

AGENTIC MARKETER

An AI-Powered Content Marketing Solution for Android



Final Year Project Report

Bachelor of Science in Software Engineering (BSSE)

Session (2022 - 2026)

Submitted By:

Aneeq Rabbani	Ali Hassan
Roll No. 12302	Roll No. 12300

Supervisor:

Dr Asim Zeb

Department of Computer Science

ABBOTTABAD UNIVERSITY OF SCIENCE & TECHNOLOGY

APPROVAL SHEET

This to certify that **Aneeq Rabbani (12302) and Ali Hassan (12300)**
has worked on and completed his/her/their Project at Department of Computer Science,
Abbottabad University of Science & Technology, Havellian Abbottabad in partial fulfillment
of the requirement for the degree of BSCS/BSSE under the guidance and supervision of
Dr Asim Zeb .

In our opinion, it is satisfactory and up to the mark and therefore fulfills the requirements of
BSCS/BSSE.

Supervisor

Department of Computer Science

(Signature)

FYP Coordinator/ Internal Examiner
Mr. Muhammad Sibte ul Hassan
Lecturer, Department of Computer Science
Abbottabad UST

(Signature)



Departmental Exam Controller
Mr. Fahad Ali Khan
Lecturer, Department of Computer Science
Abbottabad UST

(Signature)



External Examiner
Mr. Jameel Arif
Lecturer, Cyber Security Department
Air University Islamabad

(Signature)

Head of Department
Dr. Asim Zeb
Department of Computer Science
Abbottabad UST

(Signature)

DECLARATION

We, Aneeq Rabbani and Ali Hassan, hereby declare that this Final Year Project report entitled "Agentic Marketer: An AI-Powered Content Marketing Solution for Android" is our own original work. The information and content presented in this document are the result of our own research, development efforts, and intellectual contributions.

We further declare that this work has not been submitted previously, in whole or in part, to any other university or institution for the award of any degree or academic qualification. All sources of information and materials used in the preparation of this report have been duly acknowledged and referenced.

We understand that any act of plagiarism, misrepresentation, or unauthorized assistance violates academic integrity and may result in disciplinary action in accordance with the policies of Abbottabad University of Science & Technology.

Aneeq Rabbani

Roll No. 12302

Ali Hassan

Roll No. 12300

Date: _____

DEDICATION

This project is dedicated to our parents, whose prayers and unwavering support have been our greatest strength throughout every step of this journey. Their belief in our potential, even when we doubted ourselves, gave us the courage to take on a project as ambitious as this one. They sacrificed so that we could have the opportunities we do today, and for that, we are forever grateful.

We also dedicate this work to our teachers at Abbottabad University of Science & Technology, whose guidance and mentorship shaped not just this project but our entire approach to learning and problem-solving. Their patience, encouragement, and willingness to share their knowledge have been the driving force behind the successful completion of this extensive research and development endeavor.

ACKNOWLEDGEMENT

We sincerely thank our project supervisor, Dr. Asim Zeb, for his invaluable support and guidance. His technical expertise during weekly review meetings was crucial to addressing the challenges of integrating multiple AI APIs into our Android app. When we faced difficulties with Firebase Authentication setup and API rate limits, his advice guided us toward effective solutions.

We thank our FYP coordinator and all faculty members in the Department of Computer Science for providing the essential technical foundation for this application. Courses in software engineering, mobile application development, and database systems were especially useful during the project's design and implementation stages.

We also thank our friends and colleagues for their ongoing support and for serving as test users who provided honest feedback on the application's interface and functionality.

Additionally, we thank the open-source communities and the developers of Android, Kotlin, Google Gemini, and Hugging Face. Their detailed documentation and powerful APIs enabled the creation of "Agentic Marketer." The extensive tutorials, forum discussions, and official guides available online for free greatly sped up our development.

PREFACE

This project thesis details the creation of "Agentic Marketer: An AI-Powered Content Marketing Solution for Android." It discusses the different stages involved in developing this project, including:

Chapter 1: Introduction— This chapter offers an overview of the project, covering the content marketing challenges faced by small businesses, the objectives of developing Agentic Marketer, the range of features included and excluded, key technical terms definitions, detailed use case scenarios with corresponding tables, and the project plan shaped by the Work Breakdown Structure and Gantt Chart.

Chapter 2: Existing System— This chapter outlines the current state of AI-powered content marketing tools, reviews relevant academic and industry literature on generative AI in marketing, and examines the limitations of existing systems that Agentic Marketer seeks to improve.

Chapter 3: Proposed System— This chapter outlines the Agentic Marketer app's system architecture, MVVM-based modules, functional/non-functional requirements, feasibility analysis, development approach, and team structure.

Chapter 4: System Design— This chapter describes the design phase of the project, including the MVVM architectural pattern, class diagram, activity diagrams for major user flows, data flow diagrams at both context and detailed levels, sequence diagrams for authentication and content creation, the entity relationship diagram, and the graphical user interface design with screen descriptions.

Chapter 5: System Testing— This chapter outlines the thorough testing strategy used, covering black-box and white-box testing methods; detailed test cases for authentication, content generation, history management, and API error handling; and the results and discussion of the testing process.

Chapter 6: Conclusion and Future Work—This chapter summarizes the project's main accomplishments, reviews the development journey, and suggests possible future improvements, including social media integration, video creation, analytics, and collaborative tools.

ABSTRACT

Agentic Marketer is an Android app that uses Generative AI to make digital marketing easier. It addresses the problem of high costs and time needed for content creation, especially for small businesses and solo marketers who can't afford professional agencies or several premium tools.

By integrating advanced AI models—specifically Google Gemini 1.5 Pro for complex text generation and Hugging Face FLUX.1-schnell for quality image creation—the app offers a unified, mobile-centric platform. It allows users to create SEO-optimized blog posts, engaging social media captions with relevant hashtags, and professional marketing visuals. Its architecture follows the Model-View-ViewModel (MVVM) pattern, developed in Kotlin with XML layouts adhering to Material Design 3 standards. The app utilizes Firebase Authentication for secure user login and Cloud Firestore to save generated content.

The project was built using the Incremental Development Model through three iterations, enabling early feedback and step-by-step feature addition. Both black-box and white-box testing confirmed functionality, with text generation taking 5–7 seconds and image generation 15–20 seconds on stable networks. This thesis covers the entire project lifecycle—from identifying the initial problem and reviewing the literature to designing, implementing, testing, and deploying the system—demonstrating how agentic AI can serve as an effective "marketer in your pocket" for businesses with limited resources.

TABLE OF CONTENTS

CHAPTER 1	13
INTRODUCTION	13
1.1 Introduction.....	14
1.2 Purpose.....	14
1.3 Scope of the Project	15
1.4 Definitions, Acronyms, and Abbreviations	16
1.5 Use Case and Usage Scenario	17
1.5.1 Use-Case.....	17
1.5.2 Usage Scenarios	18
1.6 WBS and Gantt Chart	24
1.6.1 Work Breakdown Structure (WBS).....	24
1.6.2 Gantt Chart	25
CHAPTER 2	26
EXISTING SYSTEM	26
2.1 Existing System Overview	27
2.2 Literature Review: The Evolution of AI in Content Marketing	27
2.3 Limitations of Existing System.....	29
2.3.1 Platform Constraints and Accessibility	29
2.3.2 Cost Barrier and Subscription Models	29
2.3.3 Fragmented Workflow and Integration Challenges	30
2.3.4 Limited Customization and Brand Voice Consistency	30
CHAPTER 3	31
PROPOSED SYSTEM.....	31
3.1 Introduction to Agentic Marketer.....	32
3.2 System Architecture and Modules	32
3.2.1 Authentication Module	32
3.2.2 Content Engine Module.....	33
3.2.3 History Module.....	33
3.2.4 User Interface (UI) Module	34
3.3 Functional Requirements	34
3.4 Non-Functional Requirements	35
3.4.1 Usability	35
3.4.2 Performance.....	36
3.4.3 Reliability	36
3.4.4 Security.....	36

3.4.5 Maintainability	37
3.5 Feasibility Analysis.....	37
3.5.1 Economic Feasibility	37
3.5.2 Operational Feasibility	37
3.5.3 Technical Feasibility.....	38
3.6 Chosen Methodology: Incremental Development Model	38
3.6.1 Incremental Development Model Description	38
3.6.2 Advantages of Incremental Development	40
3.6.3 Reason for Chosen Methodology	41
3.7 Project Structure and Team Organization	41
CHAPTER 4	42
SYSTEM DESIGN	42
4.1 Introduction to System Design.....	43
4.2 Purpose of the Design Phase.....	43
4.3 Scope of the Design	43
4.4 Architectural Pattern: Model-View-ViewModel (MVVM)	44
4.5 Class Diagram.....	45
4.6 Activity Diagrams	46
4.6.1 Activity Diagram — User Login.....	46
4.6.2 Activity Diagram — Content Generation.....	48
4.7 Data Flow Diagram.....	50
4.7.1 DFD Level 0 (Context Diagram).....	50
4.7.2 DFD Level 1	50
4.8 Sequence Diagrams.....	51
4.8.1 Sequence Diagram — User Login.....	51
4.8.2 Sequence Diagram — Blog Generation	53
4.9 Entity Relationship Diagram (ERD).....	55
4.10 Graphical User Interface (GUI) Design	55
4.10.1 Splash Screen	56
4.10.2 Login / Registration Screen.....	57
4.10.3 Dashboard Screen.....	58
4.10.4 Blog Generation Screen.....	59
4.10.5 Social Media Caption Screen	60
4.10.6 Image Generation Screen	61
4.10.7 History Screen	62
CHAPTER 5	63
SYSTEM TESTING.....	63
5.1 Testing Introduction	64
5.2 Purpose of Testing.....	64

5.3 Objectives of Testing	64
5.4 Testing Techniques.....	65
5.4.1 Black Box Testing	65
5.4.2 White Box Testing.....	65
5.5 Black Box Test Cases.....	66
5.6 White Box Test Cases	67
5.6.1 White Box Test Case: API Error Handling in Content Repository.....	67
5.6.2 White Box Test Case: Firestore Data Persistence.....	69
5.7 Testing Results and Discussion.....	71
CHAPTER 6	72
CONCLUSION & FUTURE WORK	72
6.1 Conclusion	73
6.2 Future Work	74
6.2.1 Advanced Social Media Integration	74
6.2.2 Video Content Generation	74
6.2.3 Analytics and Performance Tracking.....	74
6.2.4 Multimodal Input and Generation	75
6.2.5 Personalization and Brand Consistency	75
6.2.6 Advanced Collaboration Features.....	75
REFERENCES	76

LIST OF FIGURES

Figure 1: Use Case Diagram for Agentic Marketer	18
Figure 2: Work Breakdown Structure	24
Figure 3: Gantt Chart	25
Figure 4: Incremental Development Lifecycle Model.....	39
Figure 5: Activity Diagram — User Login	47
Figure 6: Activity Diagram — Content Generation.....	49
Figure 7: DFD Level 0 (Context Diagram).....	50
Figure 8: DFD Level 1	51
Figure 9: Sequence Diagram — User Login.....	52
Figure 10: Sequence Diagram — Blog Generation	54
Figure 11: Entity Relationship Diagram	55
Figure 12: Splash Screen	57
Figure 13: Login / Registration Screen	57
Figure 14: Dashboard Screen.....	58
Figure 15: Blog Generation Screen.....	59
Figure 16: Social Media Caption Screen	60
Figure 17: Image Generation Screen	61
Figure 18: History Screen	62

LIST OF TABLES

Table 1: Definitions, Acronyms, and Abbreviations	17
Table 2: Usage Scenario — User Login / Registration (UC-01)	19
Table 3: Usage Scenario — Blog Post Generation (UC-02).....	20
Table 4: Usage Scenario — Social Media Caption Generation (UC-03)	21
Table 5: Usage Scenario — Marketing Image Generation (UC-04).....	22
Table 6: Usage Scenario — View & Manage Content History (UC-05)	23
Table 7: Black Box Test Cases.....	67
Table 8: White Box Test Cases — API Error Handling.....	69
Table 9: White Box Test Cases — Firestore Data Persistence.....	71

CHAPTER 1

INTRODUCTION

1.1 Introduction

Small businesses and independent marketers are increasingly challenged in the digital era. The demand for consistent, high-quality content across various platforms has risen sharply, yet resources for content creation remain scarce. A local bakery, a freelance consultant, or a startup e-commerce shop all require blog posts, social media captions, and marketing images to maintain visibility. However, hiring a marketing agency can cost hundreds of dollars per month, and doing it yourself requires time and skills that many small business owners lack.

The rise of Large Language Models (LLMs) and generative AI has significantly changed this landscape. Tools such as Google Gemini 1.5 Pro can quickly produce well-organized articles, while image generation models like Hugging Face FLUX.1-schnell can create marketing visuals from simple text prompts. Nonetheless, many of these tools are web-based, incur subscription costs, have scattered workflows, and offer limited mobile access. A small business owner using a smartphone cannot easily switch between a blog generator, an image creator, and a social media scheduler across different browser tabs.

Agentic Marketer was built to bridge this gap. It is a native Android application that brings text generation, image synthesis, and content history management into a single, unified mobile experience. By integrating Gemini 1.5 Pro for text and FLUX.1-schnell for images behind a clean Material Design 3 interface, the app gives users a "marketer in their pocket" — one that understands their needs, generates content accordingly, and keeps everything organized for future access.

1.2 Purpose

Agentic Marketer's main goal is to make professional content creation tools accessible through a user-friendly, mobile-focused app. It targets four key objectives. Firstly, it seeks to cut down the time needed to produce marketing content — turning hours of drafting and editing into seconds. Users can enter a topic, choose a tone, and get a full, SEO-optimized blog post directly in the app.

Second, the project aims to reduce operational costs for small businesses. Instead of paying for multiple premium services or hiring external help, users can create blogs,

social media captions with hashtags, and marketing images on a single free platform that utilizes free tiers of Firebase, Gemini, and Hugging Face APIs.

Third, the app enhances content quality with advanced AI models. Google Gemini 1.5 Pro provides sophisticated reasoning and an extensive context window for coherent, relevant texts, whereas FLUX.1-schnell generates diverse, creative visuals ideal for professional marketing.

Fourth, Agentic Marketer offers a seamless workflow that consolidates functions typically spread across multiple solutions. Users can create text and images within a single app, view their entire content history via Cloud Firestore, and share content directly with other applications on their device. This integration minimizes the inconvenience of switching tools and manually transferring content.

1.3 Scope of the Project

The scope of Agentic Marketer is defined by its core functionalities tailored specifically for the Android ecosystem. The following features are within the scope of this project:

User Authentication: Secure user registration, login, and session management utilizing Firebase Authentication with support for email/password and Google Sign-In.

AI Blog Generation: A module that accepts user topics and desired tones to generate comprehensive, SEO-friendly blog articles via the Google Gemini 1.5 Pro API.

Social Media Caption Generation: Specialized prompt handling to create engaging captions and relevant hashtags optimized for platforms such as Instagram, LinkedIn, and Twitter.

AI Image Generation: Integration with the Hugging Face FLUX.1-schnell Inference API for generating high-quality marketing visuals based on user-provided textual descriptions.

History Management: Persistent storage of all generated content (text and image URLs) using Cloud Firestore, allowing users to retrieve, review, and reuse past marketing assets.

User Interface: A responsive, accessible, and aesthetically pleasing interface adhering to Android Material Design 3 guidelines, with XML layouts enhanced by Lottie animations for a polished user experience.

The following features are explicitly out of scope for the current version:

Video Generation: The creation of video content is excluded due to API limitations, mobile processing constraints, and the significant additional complexity it would introduce.

Direct Social Media Posting: While the app generates content optimized for social platforms, direct API-based publishing to accounts (auto-posting) is deferred to future iterations.

Cross-Platform Support: The current implementation targets only native Android (Kotlin). iOS and web versions are not within the current scope.

1.4 Definitions, Acronyms, and Abbreviations

The following table defines key terms and abbreviations used throughout this thesis:

ABBREVIATION	DEFINITION
FYP	Final Year Project
AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
MVVM	Model-View-ViewModel
API	Application Programming Interface
SDK	Software Development Kit
UI/UX	User Interface / User Experience
SEO	Search Engine Optimization
JSON	JavaScript Object Notation
REST	Representational State Transfer
SME	Small and Medium Enterprise
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
WBS	Work Breakdown Structure

ABBREVIATION	DEFINITION
IDE	Integrated Development Environment
Firebase	Google's mobile platform for app development
Firestore	Cloud Firestore — NoSQL document database by Google
FLUX	Text-to-image generation model by Black Forest Labs
CNN	Convolutional Neural Network

Table 1: Definitions, Acronyms, and Abbreviations

1.5 Use Case and Usage Scenario

1.5.1 Use-Case

The Use Case Diagram shows how the authenticated user interacts with different parts of the Agentic Marketer app. It emphasizes the user's key role in starting content requests, managing their account, and checking past content. The backend systems—Firebase Authentication, Google Gemini API, and Hugging Face API—support these functions through the app's layered structure.

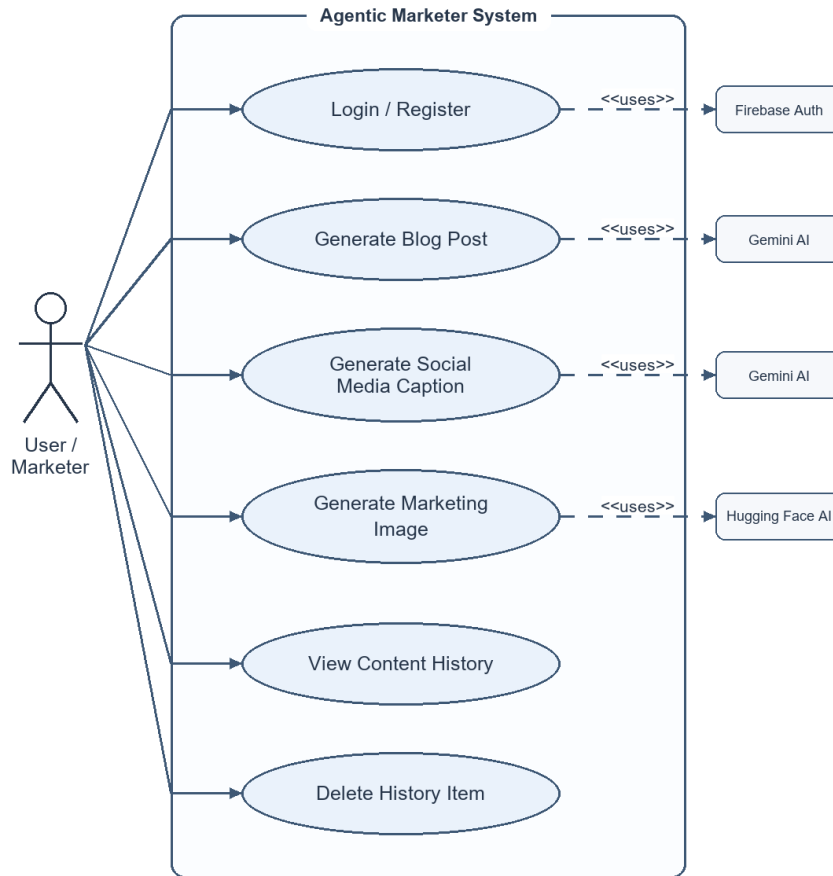


Figure 1: Use Case Diagram for Agentic Marketer

1.5.2 Usage Scenarios

The following tables describe the detailed usage scenarios for each major use case in the system.

Table 2: Usage Scenario — User Login / Registration (UC-01)

Field	Details
Use Case Title	User Login / Registration
Use Case ID	UC-01
Requirement ID	FR-01
Description	A user creates a new account or logs in to an existing account to access the application's features.
Pre-Condition	1. User must have the app installed on an Android device. 2. Device must have an active internet connection. 3. For registration, the email address must not already be associated with an existing account.
Task Sequence	1. User opens the app and views the login screen. 2. User enters email and password, or taps 'Sign in with Google'. 3. For new users, the user taps 'Create Account' and fills in registration details. 4. User taps the Login or Register button.
Task Execution	1. The Authentication ViewModel validates input format. 2. AuthRepository sends credentials to Firebase Authentication. 3. Firebase verifies credentials and returns an authentication token. 4. The app stores the token and navigates to the Dashboard.
Post-Condition	1. User is authenticated and redirected to the Dashboard. 2. Session token is stored securely for subsequent app launches.
Unresolved Issues	Handling Firebase authentication failures due to network interruptions.
Authority	All Users

Table 3: Usage Scenario — Blog Post Generation (UC-02)

Field	Details
Use Case Title	Blog Post Generation
Use Case ID	UC-02
Requirement ID	FR-02
Description	User generates an SEO-optimized blog post on a specific topic with a selected tone.
Pre-Condition	1. User must be authenticated and logged into the application. 2. Device must have an active internet connection.
Task Sequence	1. User navigates to the Blog Generator section from the dashboard. 2. User inputs the desired topic. 3. User selects the tone (Professional, Casual, or Persuasive). 4. User taps the 'Generate' button.
Task Execution	1. The ViewModel constructs a structured prompt incorporating the topic and tone. 2. The ContentRepository makes an asynchronous REST API call to the Gemini 1.5 Pro endpoint via Retrofit. 3. The system parses the JSON response and updates the UI state. 4. Generated content is automatically saved to Cloud Firestore.
Post-Condition	1. The generated blog is displayed on the screen. 2. The content is saved to the user's history in Cloud Firestore. 3. The user can copy, share, or humanize the generated text.
Unresolved Issues	Handling extremely long prompts that may exceed API token limits.
Authority	Authenticated User

Table 4: Usage Scenario — Social Media Caption Generation (UC-03)

Field	Details
Use Case Title	Social Media Caption Generation
Use Case ID	UC-03
Requirement ID	FR-03
Description	User generates engaging social media captions and relevant hashtags for a selected platform.
Pre-Condition	1. User must be authenticated. 2. Active internet connection required.
Task Sequence	1. User navigates to the Social Post Generator. 2. User selects target platform (Instagram, LinkedIn, etc.). 3. User enters the post topic or product description. 4. User taps 'Generate Caption'.
Task Execution	1. ViewModel builds a platform-specific prompt. 2. ContentRepository calls Gemini 1.5 Pro API. 3. Response is parsed to extract caption and hashtags. 4. Content is displayed and auto-saved to history.
Post-Condition	1. Caption and hashtags are displayed. 2. Content is saved to history. 3. The user can copy or share the caption.
Unresolved Issues	Platform character limits may require post-processing truncation.
Authority	Authenticated User

Table 5: Usage Scenario — Marketing Image Generation (UC-04)

Field	Details
Use Case Title	Marketing Image Generation
Use Case ID	UC-04
Requirement ID	FR-04
Description	User generates a professional marketing image based on a text description.
Pre-Condition	1. User must be authenticated. 2. Active internet connection required.
Task Sequence	1. User navigates to the Image Generator section. 2. User inputs a descriptive prompt. 3. User taps 'Generate Image'.
Task Execution	1. The system applies light automatic prompt enhancement for better results. 2. ContentRepository sends the prompt to the Hugging Face FLUX.1-schnelle API. 3. The API processes and returns the image data. 4. The application decodes and renders the image using the Coil image loading library.
Post-Condition	1. The generated image is displayed. 2. User can save to history, download to gallery, or share via other apps. 3. A compressed thumbnail is stored in the database.
Unresolved Issues	Hugging Face free tier rate limits may occasionally cause delays.
Authority	Authenticated User

Table 6: Usage Scenario — View & Manage Content History (UC-05)

Field	Details
Use Case Title	View & Manage Content History
Use Case ID	UC-05
Requirement ID	FR-08, FR-09
Description	User views previously generated content and manages their history.
Pre-Condition	1. User must be authenticated. 2. User must have previously generated at least one piece of content.
Task Sequence	1. User navigates to the History section from the dashboard. 2. The user can scroll through past content items. 3. User can filter by type (Blogs, Captions, Images). 4. The user can delete individual items.
Task Execution	1. HistoryViewModel queries FirestoreManager for the user's content collection. 2. Results are retrieved and sorted by timestamp. 3. LiveData updates the RecyclerView adapter. 4. Delete operations remove items from both the UI and Firestore.
Post-Condition	1. A list of historical content is displayed with type badges and timestamps. 2. Deleted items are removed from the list and database.
Unresolved Issues	Pagination for users with very large content histories .< join >Pagination for users with very large content histories.
Authority	Authenticated User

1.6 WBS and Gantt Chart

1.6.1 Work Breakdown Structure (WBS)

The Work Breakdown Structure decomposes the Agentic Marketer project into manageable phases and tasks. It provides a hierarchical view of the project deliverables, starting from initial planning and requirements gathering, moving through system design and core development, and concluding with rigorous testing and documentation. The WBS ensures that no critical task is overlooked and that dependencies between phases are clearly understood before development begins.

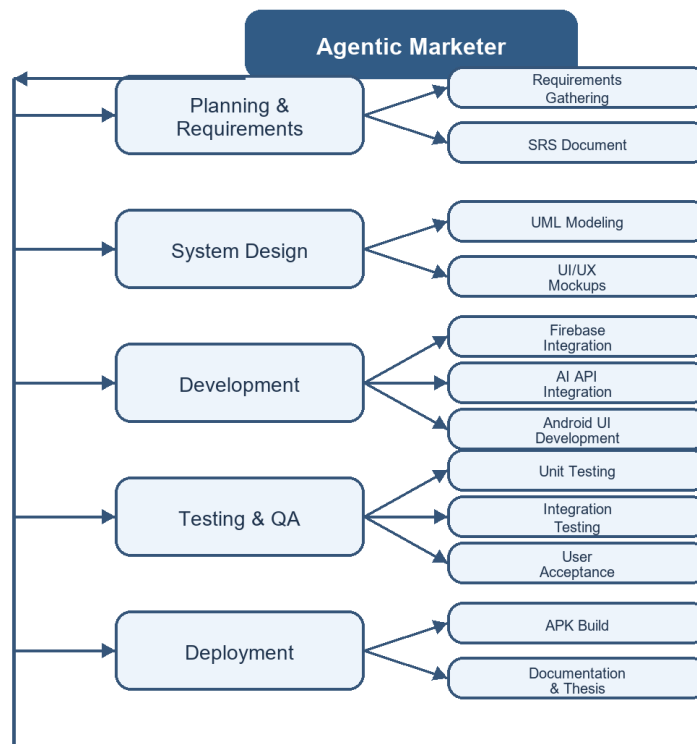


Figure 2: Work Breakdown Structure

The project was divided into five major phases:

Phase 1 — Planning & Requirements: Literature review, competitor analysis, requirement gathering, and technical specification documentation.

Phase 2 — System Design: Architecture design using MVVM pattern, database schema design, API integration planning, and UI/UX wireframing.

Phase 3 — Development: Three incremental iterations covering authentication, text generation, image generation, and UI polish with Material Design 3.

Phase 4 — Testing & QA: Black box and white box testing, API integration testing, performance benchmarking, and bug fixes.

Phase 5 — Deployment & Documentation: Final APK generation, thesis documentation, presentation preparation, and project submission.

1.6.2 Gantt Chart

The Gantt Chart provides a chronological timeline for executing the tasks defined in the WBS. It illustrates the dependencies among phases, ensuring that foundational elements such as Firebase setup and API integration are completed before advanced UI development and integration testing commence. The timeline spans the 2025–2026 academic year, with foundational tasks completed in the earlier months to allow sufficient time for testing and refinement.

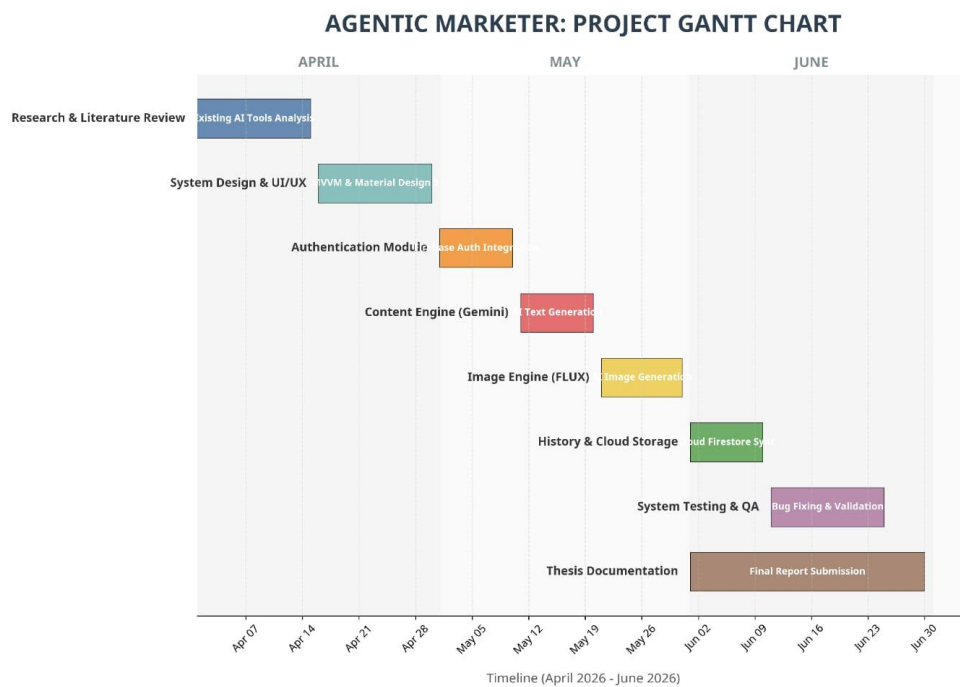


Figure 3: Gantt Chart

CHAPTER 2

EXISTING SYSTEM

2.1 Existing System Overview

The current landscape of digital marketing is heavily influenced by a diverse array of tools and platforms, many of which now incorporate Artificial Intelligence to varying degrees. These systems can be broadly categorized into general-purpose AI assistants and specialized marketing AI tools. General-purpose AI, such as OpenAI's ChatGPT and Google's Bard (now integrated into Gemini), offers broad content-generation capabilities ranging from drafting emails to summarizing complex articles. Specialized marketing AI tools, on the other hand, focus on specific aspects of the marketing funnel, including SEO optimization (e.g., Surfer SEO), social media management (e.g., Hootsuite with AI features), and graphic design (e.g., Canva with AI image generation).

While these tools have significantly enhanced productivity, they often operate in silos, requiring marketers to switch between multiple applications to complete a single campaign. Furthermore, most advanced AI marketing solutions are web-based, limiting their accessibility and utility for on-the-go content creation, especially for mobile-first businesses or individual entrepreneurs who may not have consistent access to a desktop computer. This gap in the market — the absence of a comprehensive, integrated, mobile-native solution — is precisely what Agentic Marketer aims to address.

2.2 Literature Review: The Evolution of AI in Content Marketing

The integration of AI into content marketing has been a transformative force, evolving from rudimentary automation to sophisticated generative capabilities. Early applications of AI in marketing focused primarily on data analysis, predictive analytics, and personalized recommendations. However, the advent of Large Language Models marked a significant paradigm shift, enabling AI to generate human-like text and thereby automating copywriting, blog post drafting, and social media updates. Recent academic and industry literature highlights several key trends that informed the design of Agentic Marketer .Recent academic and industry literature highlights several key trends that informed the design of Agentic Marketer Recent academic and industry literature highlights several key trends that informed the design of Agentic Marketer .

Recent academic and industry literature highlights several key trends that informed the design of Agentic Marketer.

1. Rise of Generative AI for Marketing Content: Studies consistently show that generative AI tools are becoming indispensable for content creation. Research by the U.S. Chamber of Commerce indicates that AI tools help small businesses create content faster and enhance marketing efficiency. Similarly, a 2026 report by Marketermilk identifies agentic AI automations and advanced LLMs, such as Claude, as essential to staying competitive in the marketing landscape. These findings validate the core premise of Agentic Marketer — that AI-powered content generation can deliver tangible benefits to resource-constrained businesses.

2. Challenges Small Businesses Encounter When Implementing AI Tools: Although the benefits are evident, small businesses face distinct hurdles in adopting AI for marketing. According to a Forbes article from May 2026, while 43% of small businesses utilize AI for marketing, many struggle to generate content that effectively drives conversions due to limited customer data and strategic integration. This highlights the importance of AI tools that are not only robust but also user-friendly and customized to the specific needs and resource limitations of SMEs. Agentic Marketer was created with these considerations in mind, emphasizing simplicity and mobile accessibility rather than a plethora of features.

3. Agentic AI Workflows: A burgeoning area of research is the concept of agentic AI, where AI systems are designed to operate autonomously, making decisions and executing tasks to achieve defined goals. McKinsey's research on reinventing marketing workflows with agentic AI suggests that these systems can significantly accelerate campaign execution and enable personalization at scale by reducing manual effort. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system.

system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. Deloitte Digital further emphasizes that agentic AI can reshape marketing into a more adaptive and optimized system. This shift from reactive tools to proactive, goal-oriented agents is a critical development that Agentic Marketer aims to embody through its unified content generation pipeline.

4. The Gap in Mobile-First AI Content Solutions: While many AI tools exist, there is a noticeable gap in comprehensive, integrated mobile-first solutions. The majority of powerful AI content generation tools are web-based, requiring desktop access or cumbersome mobile browser experiences. This limits their utility for marketers who are constantly on the move or for small businesses that primarily operate via mobile devices. The need for an intuitive, native Android application that brings advanced AI capabilities to mobile users' fingertips is evident and largely unmet by existing offerings.

2.3 Limitations of Existing System

2.3.1 Platform Constraints and Accessibility

A significant number of powerful AI content generation tools, such as Jasper.ai, Copy.ai, and even the full capabilities of ChatGPT and Gemini, are primarily designed for web browsers. This design choice often leads to a suboptimal user experience on mobile devices, characterized by non-responsive layouts, difficult navigation, and reduced functionality. For small business owners or marketers who spend a considerable amount of time away from a desktop, this presents a major accessibility barrier.

Existing mobile applications that offer AI marketing features are often limited in scope or act as companion apps to larger web platforms. They rarely provide a comprehensive, integrated suite of text- and image-generation capabilities within a single, fluid native mobile user interface. This forces users to juggle multiple applications, exporting content from one and importing it into another, thereby fragmenting the workflow and diminishing efficiency.

2.3.2 Cost Barrier and Subscription Models

Many premium AI content generation tools come with substantial monthly subscription fees, often ranging from \$20 to \$100 or more, depending on the feature set and usage

limits. While large enterprises can absorb these costs, they represent a significant financial burden for small businesses or individual marketers with limited budgets. The economic feasibility of adopting such tools becomes a critical concern, often leading to underutilization or reliance on less capable free alternatives.

Subscription models frequently involve tiered access, in which essential features such as advanced AI models, higher generation limits, or premium support are locked behind more expensive plans. This can restrict small businesses from accessing the full potential of AI, hindering their ability to compete effectively with larger entities that can afford top-tier subscriptions.

2.3.3 Fragmented Workflow and Integration Challenges

The current ecosystem often forces marketers to rely on separate tools for various content tasks: one for writing, another for images, and yet another for SEO or social media planning. This fragmented system causes inefficiencies, increases mental effort, and raises the risk of errors during content transfer between platforms. Even when using different AI tools, integration is usually manual — for example, creating a blog post in one tool and an image in another involves copying prompts, downloading images, uploading them elsewhere, and assembling everything manually.

Most existing AI tools are reactive; they respond to a single prompt. They lack the proactive, goal-oriented, and autonomous capabilities characteristic of agentic AI. This means users must constantly guide the AI through each step of the content creation process, rather than delegating a broader marketing objective to an intelligent agent capable of executing multiple sub-tasks independently.

2.3.4 Limited Customization and Brand Voice Consistency

Without sophisticated prompt engineering or fine-tuning capabilities, many AI tools tend to produce generic content that lacks a distinct brand voice. Small businesses, in particular, rely heavily on a unique brand identity to stand out, and generic AI outputs can dilute this effort. When using multiple AI tools or different prompts, maintaining a consistent tone, style, and messaging across various marketing channels becomes challenging. An integrated agentic system could potentially learn and adapt to a brand's specific guidelines, ensuring greater consistency.

CHAPTER 3

PROPOSED SYSTEM

3.1 Introduction to Agentic Marketer

Agentic Marketer is envisioned as a cutting-edge Android application that empowers small businesses and individual marketers with advanced AI-driven content creation capabilities. This system directly addresses the limitations identified in Chapter 2 by offering a unified, mobile-first platform that integrates text and image generation, thereby streamlining the digital marketing workflow. The core innovation lies in its agentic approach, in which the application serves as an intelligent assistant, capable of understanding high-level marketing objectives and autonomously executing multi-step content-generation tasks.

By leveraging Google Gemini 1.5 Pro for sophisticated text generation and Hugging Face FLUX.1-schnell for rapid, high-quality image synthesis, Agentic Marketer aims to provide a comprehensive, intuitive, and cost-effective solution for on-the-go content creation. The application wraps these powerful AI services behind a clean, accessible interface built with Material Design 3 principles, ensuring that users of all technical backgrounds can generate professional marketing content without prior expertise in AI or prompt engineering.

3.2 System Architecture and Modules

The Agentic Marketer application is built on a robust, scalable architecture, primarily adhering to the Model-View-ViewModel (MVVM) design pattern for Android development. This pattern promotes a clear separation of concerns, enhancing maintainability, testability, and scalability of the codebase. The system comprises four interconnected modules, each responsible for a specific set of functionalities, ensuring modularity and efficient resource management.

3.2.1 Authentication Module

Purpose: To provide secure user registration, login, and session management. This module is critical for personalizing user experience and protecting user data.

Technology: Firebase Authentication — a comprehensive, secure, and easy-to-integrate authentication service that supports various authentication methods.

Key features include: user registration with email and password; user login and logout functionalities; password reset and account recovery via email; secure session management and token handling; and integration with Google Sign-In for enhanced user convenience. Firebase securely handles all credential storage, ensuring that sensitive information is never stored in plain text within the application.

3.2.2 Content Engine Module

Purpose: The heart of the application, responsible for orchestrating AI model interactions to generate text and image content. This module abstracts the complexities of API calls and data parsing from the rest of the application.

Technology Stack: Google Gemini 1.5 Pro API for advanced text generation (blogs, social media captions, and other textual marketing collateral); Hugging Face FLUX.1-schnell API for rapid and high-quality image synthesis from textual prompts; Retrofit as a type-safe HTTP client for making asynchronous API calls to both endpoints.

Key features include: dynamic prompt construction based on user input (topic, tone, style); asynchronous API calls to external AI services with proper threading; robust error handling for network failures and API rate limits; parsing and formatting of AI-generated content for display in the UI; and light automatic prompt enhancement for image generation to ensure consistently professional results.

3.2.3 History Module

Purpose: To provide persistent storage and retrieval of all user-generated content, allowing users to review, reuse, and manage their past marketing assets.

Technology: Cloud Firestore — a NoSQL document database that offers real-time synchronization, offline support, and scalable storage, making it ideal for storing user-specific content history.

Key features include: automatic saving of generated text and image URLs upon successful generation; retrieval of historical content categorized by type (blog, social post, image); ability to delete past entries; real-time updates to the history list; and compressed thumbnail storage for images to avoid Firebase Storage costs while keeping full-resolution images available via the Download to Gallery feature. During Increment 2,

the team discovered that Firestore indexing needed adjustment to support efficient history queries on large datasets, which was resolved by adding composite indexes.

3.2.4 User Interface (UI) Module

Purpose: To provide an intuitive, responsive, and aesthetically pleasing interface for users to interact with the application, focusing on user experience and visual design.

Technology Stack: Kotlin with XML layouts; Jetpack Compose elements where declarative UI provides advantages; Material Design 3 guidelines for consistent modern aesthetics; Lottie animations for smooth loading indicators and transitions; Navigation Component for seamless screen transitions.

Key features include: a dashboard with quick-access cards for Blog, Social Post, Image Generation, and History; dedicated screens for each generation type with clear input fields and output areas; responsive design across various Android screen sizes; loading indicators and feedback mechanisms during AI generation; a visible usage tracker on the image screen showing remaining free generations; and a bottom navigation bar with the Create button centered (matching Instagram and TikTok conventions).

3.3 Functional Requirements

Functional requirements define the specific actions the system must perform to meet user needs. For an Agentic Marketer, these include:

FR-01: User Authentication — The system shall allow users to register, log in, and log out securely using email/password and Google Sign-In via Firebase Authentication.

FR-02: Blog Generation — The system shall enable users to generate SEO-optimized blog posts by providing a topic and selecting a desired tone (Professional, Casual, or Persuasive) via the Gemini 1.5 Pro API.

FR-03: Social Media Caption Generation — The system shall allow users to generate short, engaging social media captions and relevant hashtags based on a given topic or product description, optimized for the selected platform.

FR-04: Marketing Image Generation — The system shall generate high-quality marketing images from user-provided textual descriptions via the Hugging Face FLUX—1-schnell API.

FR-05: Content Display — The system shall display generated text content and images within the application interface in a readable format.

FR-06: Content Sharing — The system shall allow users to share generated text content to other applications (email, messaging apps) and save generated images to their device's gallery.

FR-07: History Auto-Save — The system shall automatically save all generated content to a user-specific history log in Cloud Firestore upon successful generation.

FR-08: History Retrieval — The system shall allow users to view and retrieve their past generated content, with filtering by content type.

FR-09: Content Deletion — The system shall provide functionality for users to delete individual items from their content history.

3.4 Non-Functional Requirements

3.4.1 Usability

NFR-U1: Intuitive UI — The application shall feature an intuitive, easy-to-navigate user interface that adheres to Android Material Design 3 guidelines, minimizing the learning curve for new users.

NFR-U2: Responsive Design — The UI shall be responsive across various Android device screen sizes and orientations.

NFR-U3: Clear Feedback — The system shall provide clear visual and textual feedback for all user actions, especially during AI content generation (loading indicators, success/error messages).

3.4.2 Performance

NFR-P1: Fast Content Generation — Text content generation shall typically complete within 5–10 seconds, and image generation within 10–20 seconds, depending on network conditions and API response times.

NFR-P2: Smooth Navigation — Transitions between screens and loading of historical content shall be smooth and lag-free.

NFR-P3: Efficient Resource Usage — The application shall optimize CPU, memory, and battery consumption to ensure a seamless user experience without excessive device drain.

3.4.3 Reliability

NFR-R1: High Availability — The system shall leverage Firebase's cloud infrastructure to ensure high availability for authentication and data storage, with a target of 99.9% uptime.

NFR-R2: Error Handling — The application shall gracefully handle API errors, network disconnections, and other exceptions, providing informative messages to the user without crashing.

NFR-R3: Data Integrity — User-generated content stored in Cloud Firestore shall maintain integrity and consistency, preventing data loss or corruption.

3.4.4 Security

NFR-S1: Secure Authentication — User credentials shall be securely handled and stored by Firebase Authentication, adhering to industry best practices for password hashing and storage.

NFR-S2: Data Privacy — User-generated content stored in Firestore shall be accessible only to the authenticated user, with appropriate access control rules in place.

NFR-S3: API Key Protection — API keys for Gemini and Hugging Face shall be securely managed and not exposed in the client-side code.

3.4.5 Maintainability

NFR-M1: Modular Codebase — The codebase shall be modular, following the MVVM architecture, to facilitate easy updates, bug fixes, and feature additions.

NFR-M2: Documentation — The code shall be well-documented with comments and clear naming conventions.

NFR-M3: Testability — Individual components of the application (ViewModels, Repositories) shall be designed to be easily testable.

3.5 Feasibility Analysis

3.5.1 Economic Feasibility

The project leverages several free-tier services, significantly reducing development and initial deployment costs. Firebase offers generous free tiers for Authentication and Firestore. Google Gemini 1.5 Pro provides free usage for development and testing within certain limits. Hugging Face also offers free inference APIs for FLUX.1-schnell within a reasonable rate limit. The development tools — Android Studio, Kotlin, and Gradle — are all freely available. The primary costs would arise only when scaling up API usage, if the user base grows significantly, and can be managed through a freemium model or subscription plans. The potential benefits (time savings for marketers, reduced content creation costs, enhanced content quality) far outweigh the minimal development and operational costs, especially for the target audience of small businesses and individual marketers.

3.5.2 Operational Feasibility

The intuitive UI/UX design, coupled with the clear value proposition of automated content creation, suggests high user acceptance. The mobile-first approach aligns with modern work habits, particularly for small business owners who manage their operations primarily through smartphones. Agentic Marketer is designed to integrate into existing marketing workflows by providing shareable content that can be copied or shared directly to social media platforms, email clients, and messaging apps. While direct social media posting is out of scope, the ease of generating content within the app and then sharing it

minimizes disruption to existing workflows. The application's simplicity and adherence to Material Design 3 guidelines mean minimal to no training will be required for typical Android users.

3.5.3 Technical Feasibility

The chosen technology stack represents modern, well-supported, and robust solutions for Android development and AI integration. Kotlin is the officially recommended language for Android development, with excellent tooling in Android Studio. Firebase provides battle-tested backend services with extensive documentation. Retrofit and OkHttp are industry-standard networking libraries. All APIs are well-documented and have active developer communities. The development team possesses the requisite skills in Kotlin, Android development, Firebase, and API integration to implement the project successfully. The application targets Android 5.0 Lollipop (API 21) and above, covering the vast majority of the Android user base.

3.6 Chosen Methodology: Incremental Development Model

3.6.1 Incremental Development Model Description

The Incremental Development Model is a hybrid approach that combines elements of the Waterfall model with the iterative nature of agile methodologies. It involves breaking the project down into smaller, manageable increments, with each increment building on the previous one and delivering a functional subset of the overall system. Each increment goes through the typical software development life cycle phases: requirements, design, implementation, and testing. After each increment, a working version of the software is delivered, enabling early stakeholder feedback and continuous refinement.

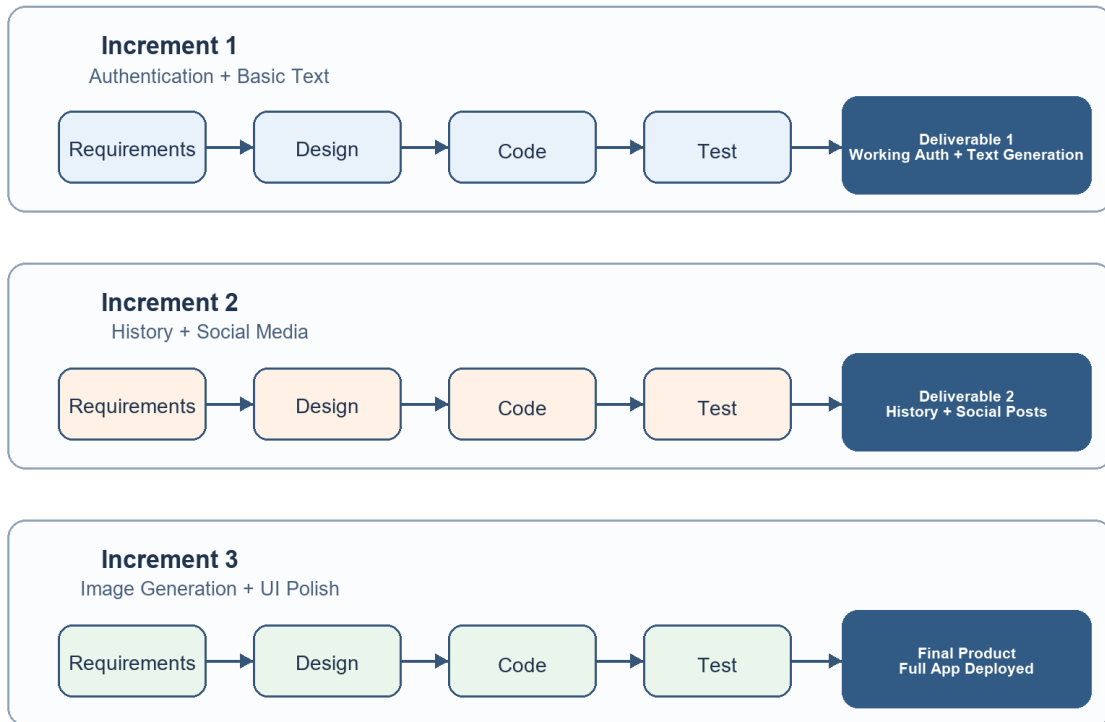


Figure 4: Incremental Development Lifecycle Model

The development of Agentic Marketer was structured into three increments:

Increment 1 — Core Authentication and Basic Text Generation: This increment focused on establishing the foundational architecture. Requirements included user registration, login, and basic blog generation (topic-based). We implemented Firebase Auth integration, established the basic MVVM structure, wrote Kotlin code for authentication flows, set up Retrofit for Gemini API calls, and built a basic UI for text input and display. The deliverable was a functional app capable of user login and generating simple text content.

Increment 2 — Enhanced Text Generation and History Management: This increment expanded text-generation capabilities and added persistence. Requirements included tone selection for text generation and social media post generation, as well as saving generated content to history. We expanded ViewModel logic for tone parameters, integrated Cloud Firestore for history storage, enhanced the UI for tone selection, created Firestore data

models, and built the history display screen. During this increment, we discovered that Firestore indexing needed adjustment for efficient history queries, which required adding composite indexes to the Firebase project. The deliverable was an app with advanced text generation and a functional history feature.

Increment 3 — Image Generation and UI Polish: This final increment added visual content generation and refined the user experience. Requirements included image generation from text prompts and improved UI/UX with full Material Design 3 theming. We integrated the Hugging Face FLUX.1-schnell API, implemented image display components using Coil, applied the complete Material 3 theme, added Lottie animations for loading states, and implemented the visible usage tracker for image generation limits. API rate limits from Hugging Face's free tier required implementing a retry mechanism with exponential backoff. The deliverable was the complete application with all core AI features and a polished, professional UI.

3.6.2 Advantages of Incremental Development

The Incremental Development Model offered several advantages for this project:

Early Delivery of Partial Functionality: Stakeholders could see and interact with a working version of the application early in the development cycle, providing valuable feedback that informed subsequent increments.

Reduced Risk: Issues were identified and addressed incrementally, reducing the risk of major failures later in the project. When the initial Gemini API integration encountered authentication challenges during Increment 1, we resolved them without delaying other features.

Flexibility: The model allowed for easier incorporation of changes and new requirements, as each increment could be adapted based on feedback from our supervisor and informal user testing.

Better Resource Management: Resources could be allocated more effectively to each increment, preventing bottlenecks in the two-person team.

Improved User Satisfaction: Continuous delivery of functional increments increased engagement and enabled us to demonstrate tangible progress during academic reviews.

3.6.3 Reason for Chosen Methodology

The Incremental Development Model was chosen for its suitability for a project involving ongoing integration with evolving external AI APIs. This approach helped manage the complexity of integrating multiple AI models into a robust Android app by breaking the work into small sections. It also enabled quick responses to API updates like Gemini and Hugging Face without halting development. Early feedback on core functionalities ensured the app stayed aligned with user needs and market trends. It also allowed delivery of working versions at different stages, useful for academic and stakeholder reviews.

3.7 Project Structure and Team Organization

The project was structured to maximize efficiency and collaboration within a lean two-person development team, with clear roles and responsibilities:

Ali Hassan— Backend Development: Focused on backend integration, including Firebase Authentication setup, Cloud Firestore configuration, API integration using Retrofit and OkHttp, ViewModel and Repository implementation, data modeling, and core business logic. Responsible for the Content Engine module and ensuring robust error handling across all API interactions.

Aneeq Rabbani— Frontend Development: Focused on frontend UI/UX development, including XML layout design, Jetpack Compose elements, Material Design 3 theming, Lottie animation integration, navigation flow design, and responsive screen implementations. Responsible for ensuring the application met modern Android design standards and provided an intuitive user experience.

The team employed a collaborative strategy, holding daily stand-ups, weekly reviews with supervisor Dr. Asim Zeb, and continuously integrating code via Git and GitHub for version control. Tasks were allocated according to each member's strengths and expertise, promoting cross-training to share knowledge. Key integration tasks—such as linking the UI to the ViewModel layer and connecting API responses to display components—were completed through pair programming to improve code quality and ensure that both team members understood the entire data flow.

CHAPTER 4

SYSTEM DESIGN

4.1 Introduction to System Design

System design plays a vital role in the software development lifecycle, transforming the high-level requirements from Chapter 3 into a detailed blueprint for building the system. This chapter discusses the architectural decisions, design patterns, and structural elements of the Agentic Marketer app. The main objective during the design phase was to develop a system that is robust, scalable, maintainable, and easy to use, seamlessly incorporating advanced AI features into a mobile platform. Focus was on modularity, separation of concerns, and following modern Android development standards to ensure the system's durability and potential for future growth.

4.2 Purpose of the Design Phase

The design phase had several important goals. Firstly, it created a detailed blueprint that guided the implementation team, helping ensure consistent and correct construction of all components. Secondly, it made sure the proposed design effectively met all functional and non-functional requirements. Thirdly, it established a modular architecture with loosely connected components, making debugging, updates, and future enhancements easier. Fourth, it prepared the system to handle higher user loads and data without major architectural changes. Fifth, it involved making design choices that enhance application performance, such as efficient data processing and API interactions. Lastly, it resulted in a user interface that is both intuitive and engaging, following modern Android design standards.

4.3 Scope of the Design

The design scope includes the full Agentic Marketer app, ranging from the user interface to backend API integrations and data storage. It involves choosing and applying the MVVM architecture, designing components for modules like Authentication, Content Engine, History, and UI, modeling data for Cloud Firestore, interacting with external AI services such as Gemini and Hugging Face, structuring layouts and navigation flows, and addressing security to protect data privacy and ensure safe operation.

4.4 Architectural Pattern: Model-View-ViewModel (MVVM)

The Model-View-ViewModel (MVVM) architectural pattern was chosen for the Agentic Marketer application due to its proven benefits in Android development, particularly with Kotlin. MVVM promotes a clear separation of concerns, making the codebase more organized, testable, and maintainable.

Model: Represents the data and business logic. In Agentic Marketer, this includes data classes for user information (User) and generated content (GeneratedContent), as well as the repository layer that handles data operations — fetching from APIs via ApiService, saving to Firestore via FirestoreManager, and caching where appropriate.

View: The UI layer, responsible for displaying data and capturing user input. This consists of Android Activities and XML layouts (with select Jetpack Compose elements) that observe LiveData from ViewModels and update the interface accordingly.

ViewModel: Acts as a bridge between the Model and the View. It exposes data streams from the Model to the View and handles UI-related logic, preparing data for display. ViewModels are lifecycle-aware, meaning they survive configuration changes (like screen rotations), preventing data loss and unnecessary re-fetching.

The advantages of MVVM for this project are significant:

Separation of Concerns: Clearly delineates responsibilities, making it easier for both team members to work on distinct parts of the application simultaneously without conflicts.

Testability: ViewModels can be tested independently of the Android UI, simplifying unit testing and improving code quality. We leveraged this extensively during the white box testing phase.

Maintainability: Changes in the UI or data layer have minimal impact on other components, reducing the risk of introducing new bugs when modifying existing features.

Scalability: The modular nature of MVVM allows for easy addition of new features and modules without significant refactoring, which proved valuable when adding image generation in Increment 3.

4.5 Class Diagram

The Class Diagram provides a static structural view of the system, illustrating the classes, their attributes, methods, and relationships. It highlights the core components of the MVVM architecture as applied in Agentic Marketer. The main classes and their responsibilities are:

MainActivity (View): The entry point of the application, responsible for setting up the UI and observing LiveData from the MainViewModel.

MainViewModel (ViewModel): Holds and processes UI-related data. It interacts with ContentRepository to fetch or generate data and exposes LiveData objects (contentResult, isLoading, errorMessage) to MainActivity.

ContentRepository (Model): Abstracts data sources. It decides whether to fetch data from remote APIs (Gemini, Hugging Face) via ApiService or from local persistence (Firestore). Handles caching and data manipulation.

GeminiApiService / HuggingFaceApiService (Model): Interfaces defined using Retrofit, specifying the API endpoints for Gemini and Hugging Face. Handle the actual HTTP requests and receive raw JSON responses.

FirestoreManager (Model): Handles all interactions with Firebase Authentication and Cloud Firestore for user management and history persistence.

User (Data Class): Represents user data, including userId, email, and name.

GeneratedContent (Data Class): Represents a piece of generated content, including contentId, userId, type (blog, social, image), topic, body, imageUrl, and timestamp.

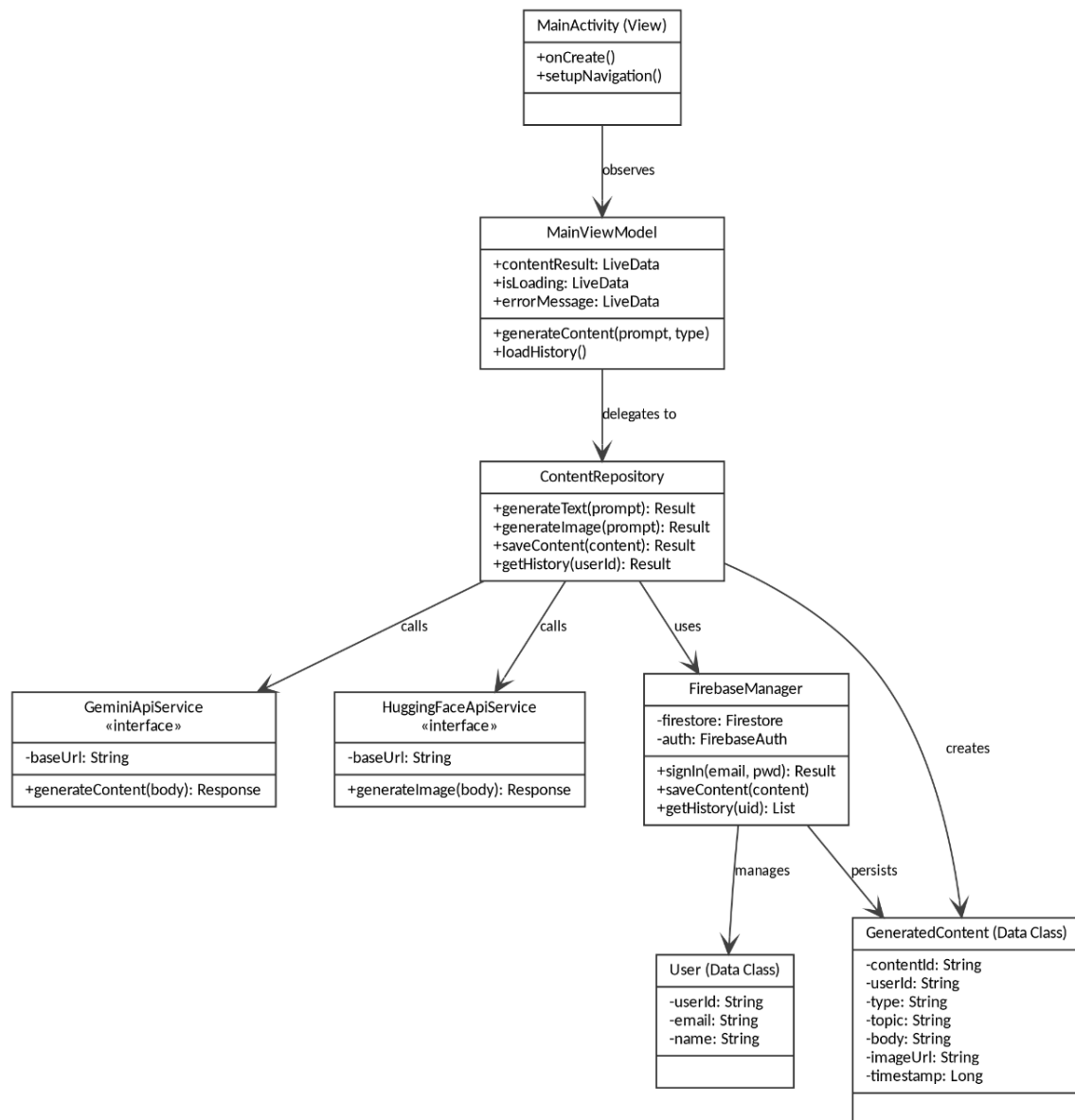


Figure 5: Class Diagram

4.6 Activity Diagrams

Activity diagrams depict the flow of control within the system, capturing dynamic elements like user interactions and background processes.

4.6.1 Activity Diagram — User Login

This diagram depicts the steps a user takes to log into the app. It begins with the user opening the app and viewing the login screen. The user then enters their email and password, which are checked for proper format. If validation passes, the credentials are

sent to Firebase Authentication. After successful authentication, the user is taken to the Dashboard. If authentication fails, an error message appears, prompting the user to re-enter their credentials. The process ends when the user either successfully accesses the Dashboard or chooses to create a new account.

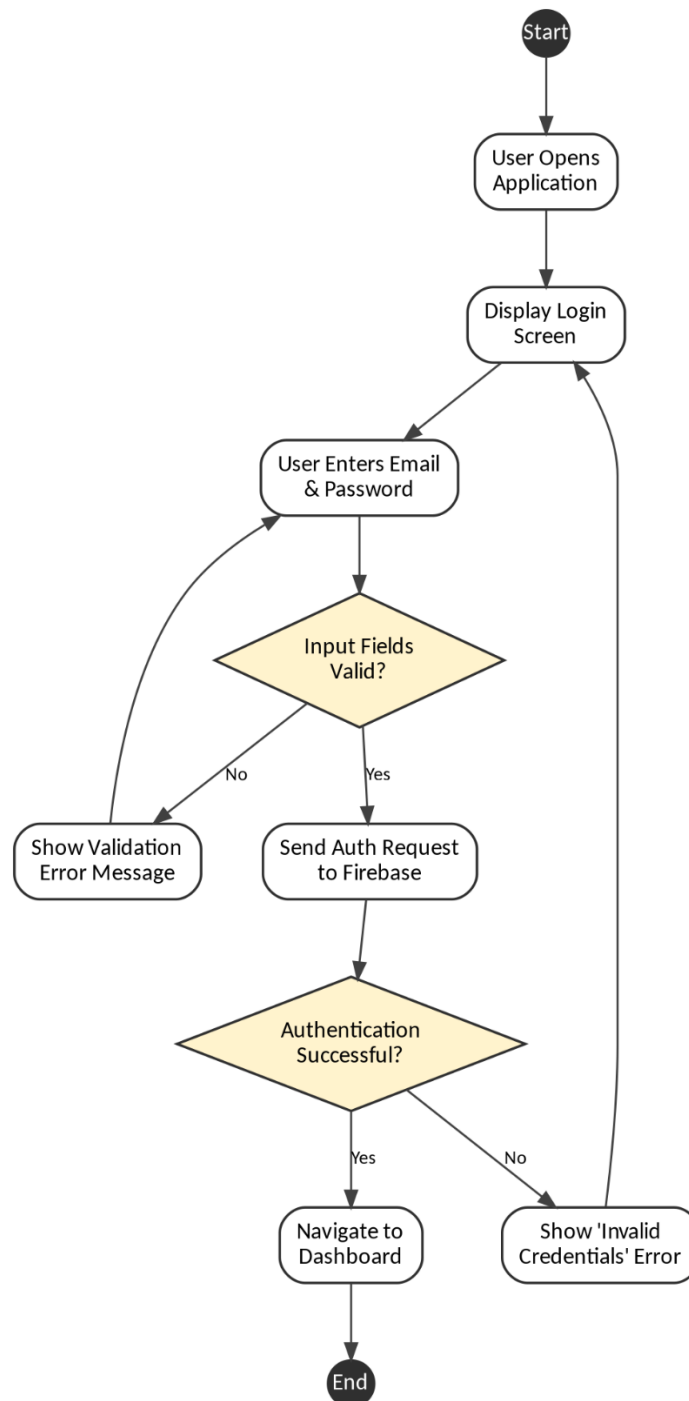


Figure 5: Activity Diagram — User Login

4.6.2 Activity Diagram — Content Generation

This diagram outlines the workflow for content creation, whether it's a blog post, social caption, or image. The process begins when the user opens a generation module and enters necessary parameters such as topic, tone, or image prompt. Upon tapping the Generate button, input validation occurs. If valid, the ViewModel creates the API request, and the Repository forwards it to the appropriate AI service (Gemini for text, Hugging Face for images). A loading indicator shows during processing. When the API responds successfully, the content is parsed, displayed, and automatically saved to Firestore history. If the request fails, an error message appears with an option to retry.

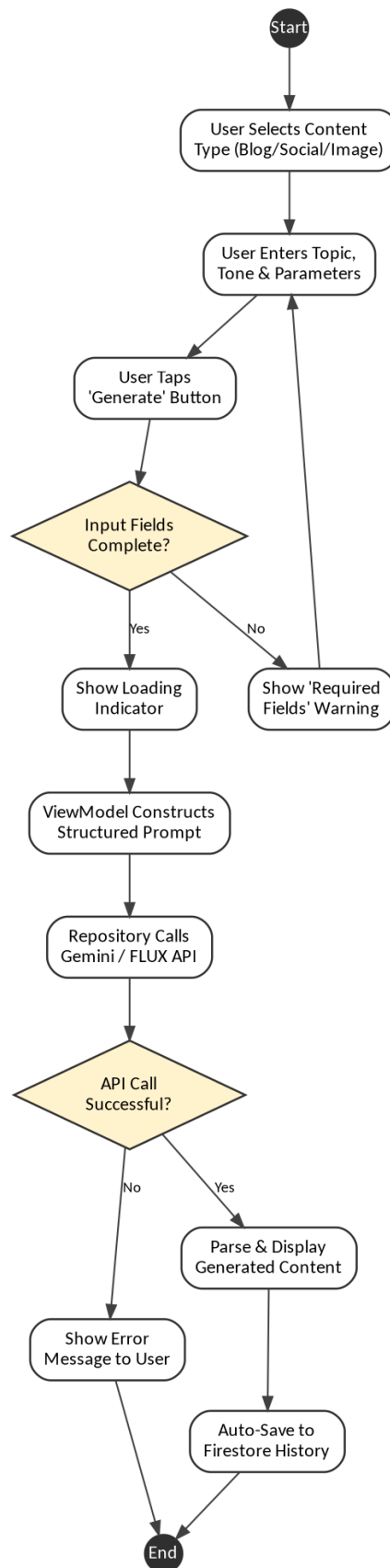


Figure 6: Activity Diagram — Content Generation

4.7 Data Flow Diagram

4.7.1 DFD Level 0 (Context Diagram)

The Level 0 DFD, or Context Diagram, represents the entire system as a single process and shows its interaction with external entities. The external entities are: User (interacts with the Android application), Firebase (provides authentication and database services), Gemini AI (provides text generation services), and Hugging Face AI (provides image generation services). Data flows include: User provides input to Agentic Marketer; Agentic Marketer sends requests to Firebase, Gemini AI, and Hugging Face AI; these services return data to Agentic Marketer; and Agentic Marketer displays output to the User.

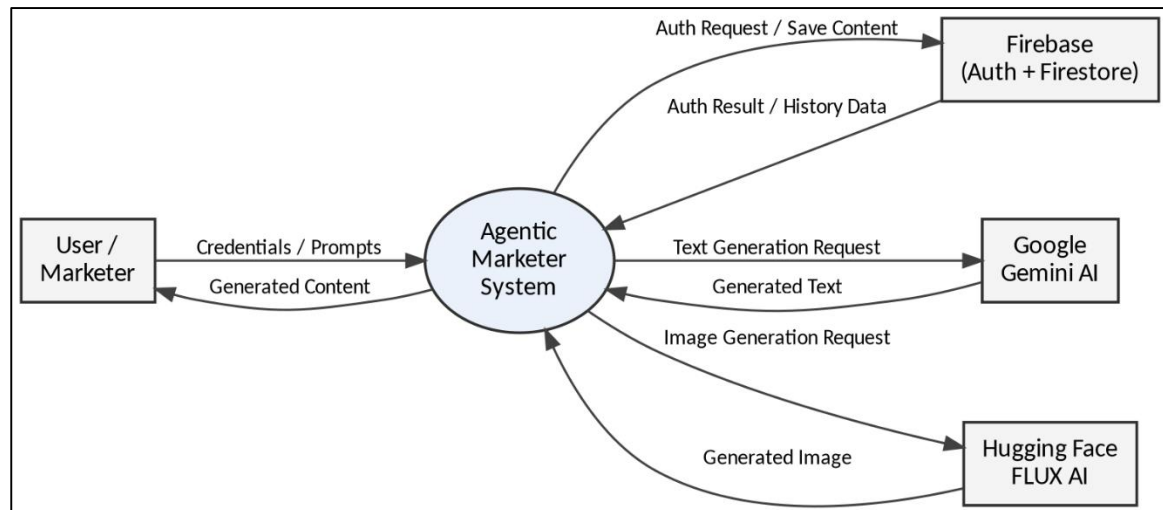


Figure 7: DFD Level 0 (Context Diagram)

4.7.2 DFD Level 1

The Level 1 DFD breaks down the main process into its major sub-processes, showing the data stores and data flows between them. The three main processes are: P1 — User Authentication (handles login/registration via Firebase Auth), P2 — Content Generation (manages AI API calls for text and image generation), and P3 — History Management (stores and retrieves generated content). The two data stores are: DS1 — Firebase Auth (user credentials), and DS2 — Cloud Firestore (generated content history). Data flows include: User provides credentials to P1, which interacts with DS1; User provides prompts to P2, which interacts with Gemini AI and Hugging Face AI; P2 sends generated

content to P3, which stores it in DS2; and User requests history from P3, which retrieves content from DS2.

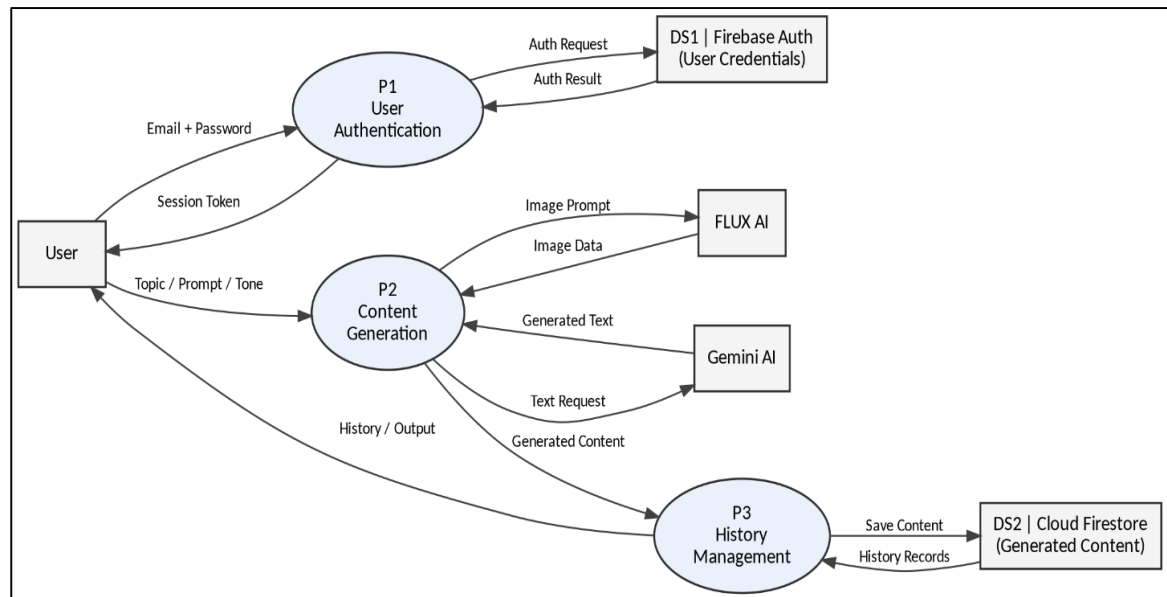


Figure 8: DFD Level 1

4.8 Sequence Diagrams

4.8.1 Sequence Diagram — User Login

This diagram details the message flow during a user login operation. The participants are: User, LoginScreen, LoginViewModel, AuthRepository, and Firebase Authentication. The flow is as follows:

- (1) User enters credentials into LoginScreen.
- (2) LoginScreen calls LoginViewModel.login(email, password).
- (3) LoginViewModel calls AuthRepository.signIn(email, password).
- (4) AuthRepository calls Firebase Authentication.signInWithEmailAndPassword(email, password).
- (5) Firebase Authentication returns AuthResult to AuthRepository.
- (6) AuthRepository returns Result<User> to LoginViewModel.

(7) LoginViewModel updates loginState LiveData.

(8) LoginScreen observes loginState and navigates to Dashboard on success or displays an error message.

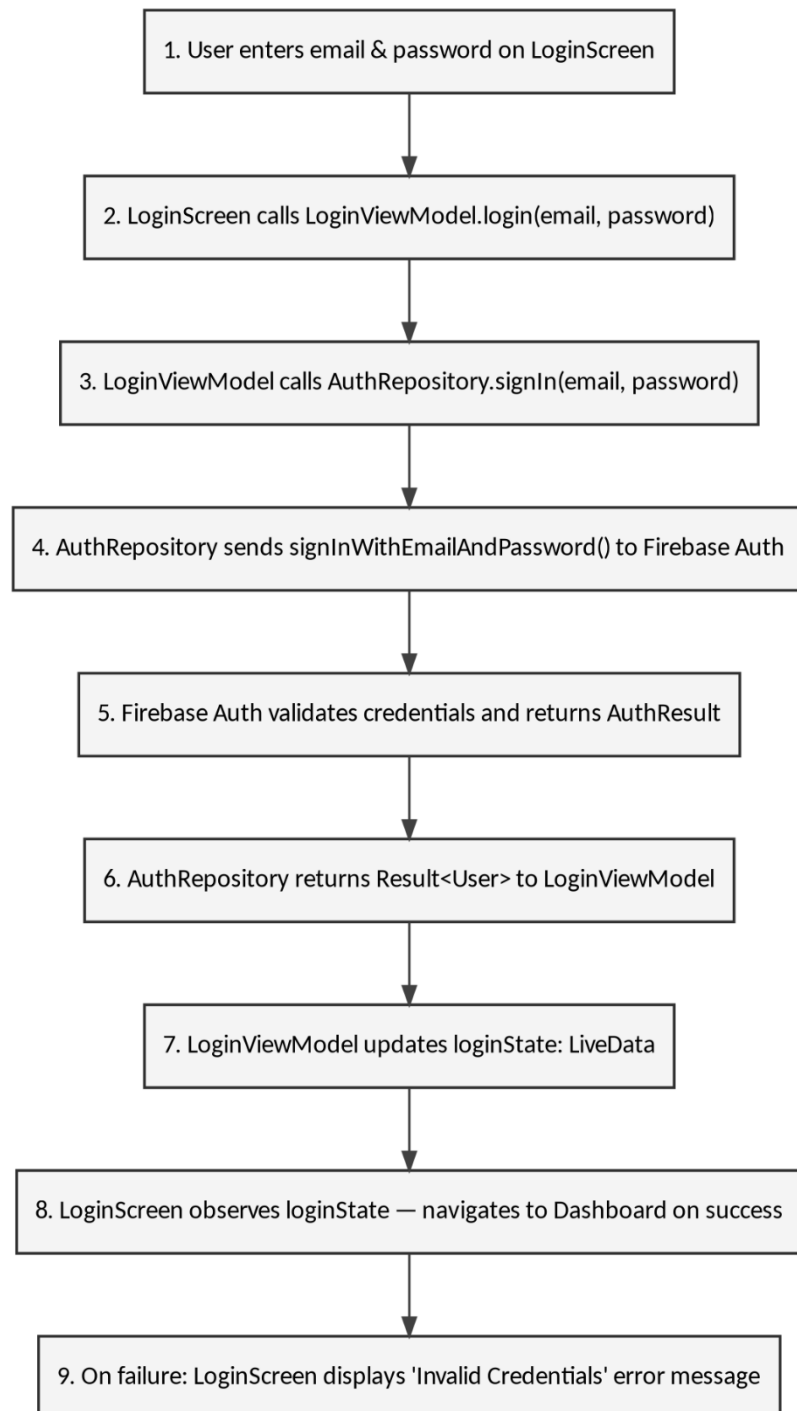
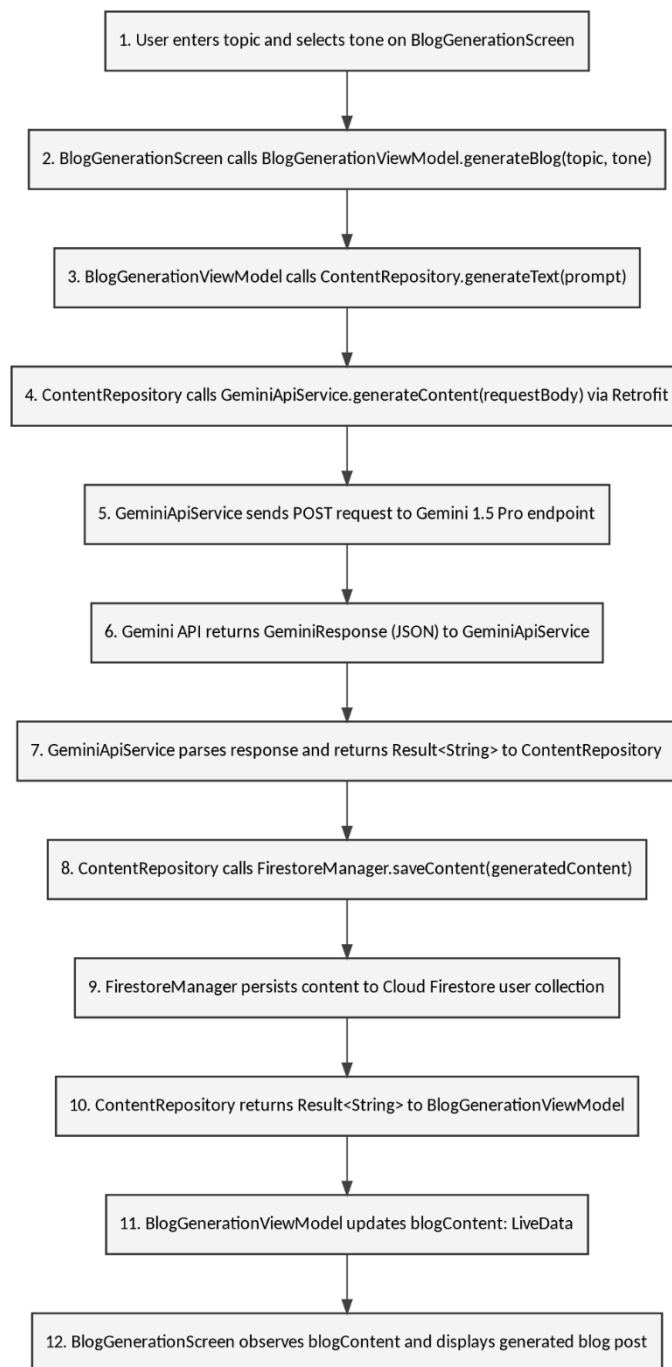


Figure 9: Sequence Diagram — User Login

4.8.2 Sequence Diagram — Blog Generation

This diagram shows the interaction flow when a user generates a blog post. The participants are: User, BlogGenerationScreen, BlogGenerationViewModel, ContentRepository, GeminiApiService, Gemini AI API, and FirestoreManager. The flow proceeds as follows:

- (1) User inputs topic and tone into BlogGenerationScreen.
- (2) BlogGenerationScreen calls BlogGenerationViewModel.generateBlog(topic, tone).
- (3) BlogGenerationViewModel calls ContentRepository.generateText(prompt).
- (4) ContentRepository calls GeminiApiService.generateContent(requestBody).
- (5) GeminiApiService sends the request to the Gemini AI API.
- (6) Gemini AI API returns the response.
- (7) GeminiApiService returns Result<String> to ContentRepository.
- (8) ContentRepository calls FirestoreManager.saveContent to persist the generated content.
- (9) FirestoreManager saves the content to Cloud Firestore.
- (10) ContentRepository returns the result to BlogGenerationViewModel.
- (11) BlogGenerationViewModel updates blogContent LiveData.
- (12) BlogGenerationScreen observes the update and displays the generated blog.

**Figure 10: Sequence Diagram — Blog Generation**

4.9 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram visually represents the relationships among entities in the database. For an Agentic Marketer, the ERD focuses on the Cloud Firestore data model, illustrating how user data and generated content are structured and linked.

The USER entity contains: `userId` (Primary Key, provided by Firebase Auth), `email`, and `name`. The CONTENT entity contains: `contentId` (Primary Key), `userId` (Foreign Key linking to USER), `type` (blog, social_post, or image), `topic`, `body` (textual content), `imageUrl` (for image content), and `timestamp` (date and time of generation). The relationship between these entities is one-to-many: one user can generate multiple pieces of content, but exactly one user generates each piece of content.

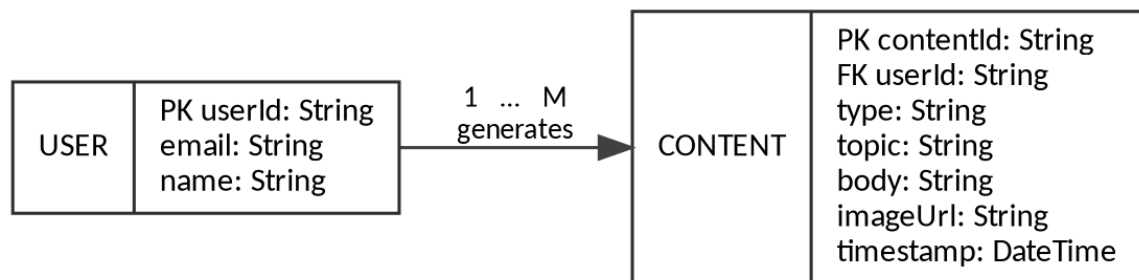


Figure 11: Entity Relationship Diagram

4.10 Graphical User Interface (GUI) Design

The Graphical User Interface is the primary point of interaction for the user. The design of Agentic Marketer prioritizes clarity, ease of use, and adherence to modern Android design principles, specifically Material Design 3. The following sections describe the key screens of the application.

4.10.1 Splash Screen

The splash screen serves as the first visual element users encounter when launching the application. It displays the Agentic Marketer logo and app name with a brief tagline ('Developed by Aneeq Rabbani and Ali Hassan'), establishing brand identity. At the same time, the application initializes in the background—a Lottie animation plays during initialization to indicate progress.



Figure 12: Splash Screen

4.10.2 Login / Registration Screen

This screen allows new users to register and existing users to log in. It features clear input fields for email and password, a prominent login button, a toggle to switch to registration mode, a Google Sign-In button for quick authentication, and a 'Forgot Password' link for account recovery. The layout follows Material Design 3 guidelines with proper spacing and elevation.

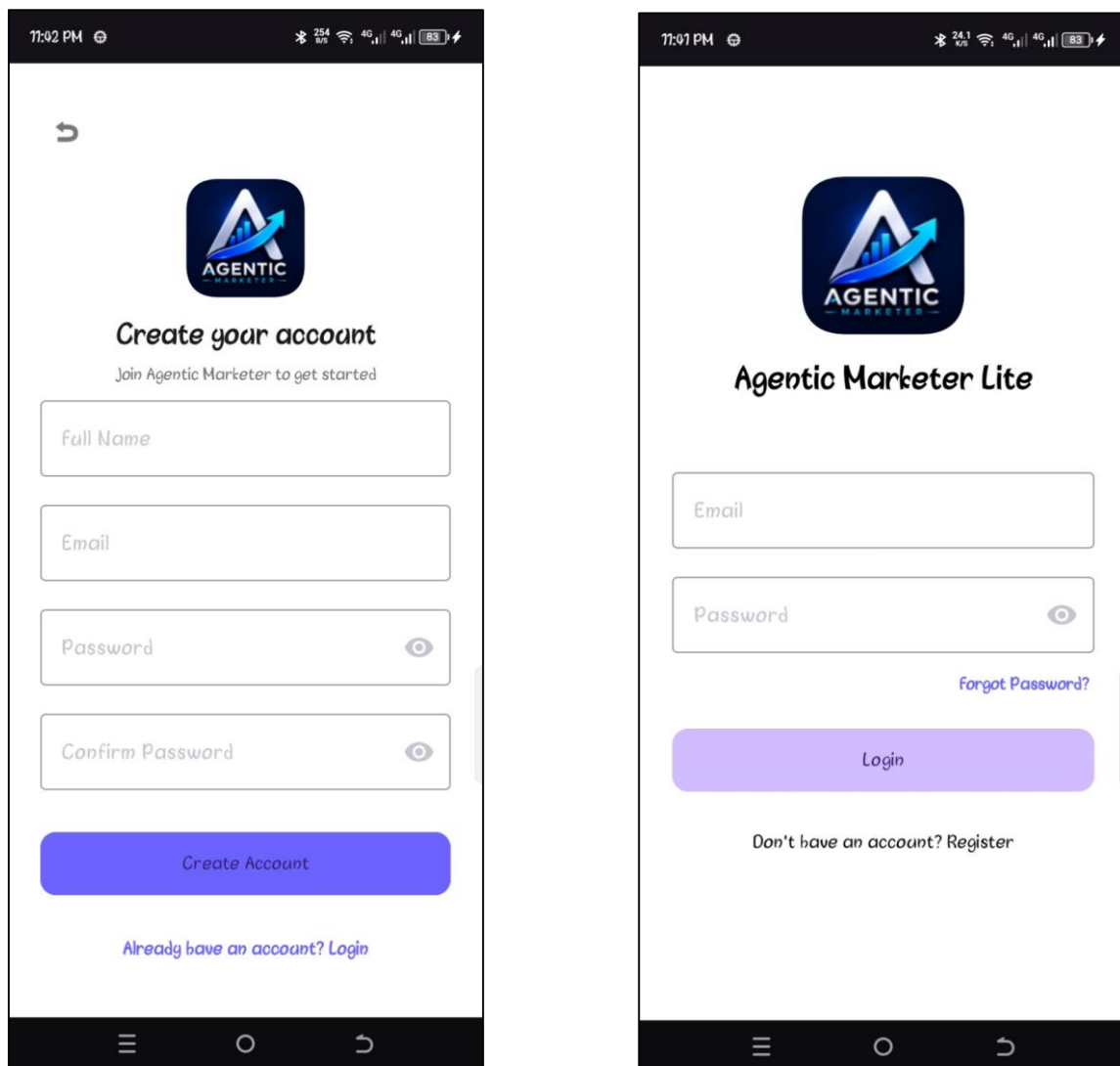


Figure 13: Login / Registration Screen

4.10.3 Dashboard Screen

The dashboard serves as the central hub of the application, providing quick access to all major features. It displays a personalized greeting (e.g., 'Good Evening, Ali'), statistics showing counts for generated Blogs, Captions, and Images, and large quick-action cards for AI Blog, Social Post, AI Image, and History. A Pro Tip section at the bottom provides helpful marketing advice.

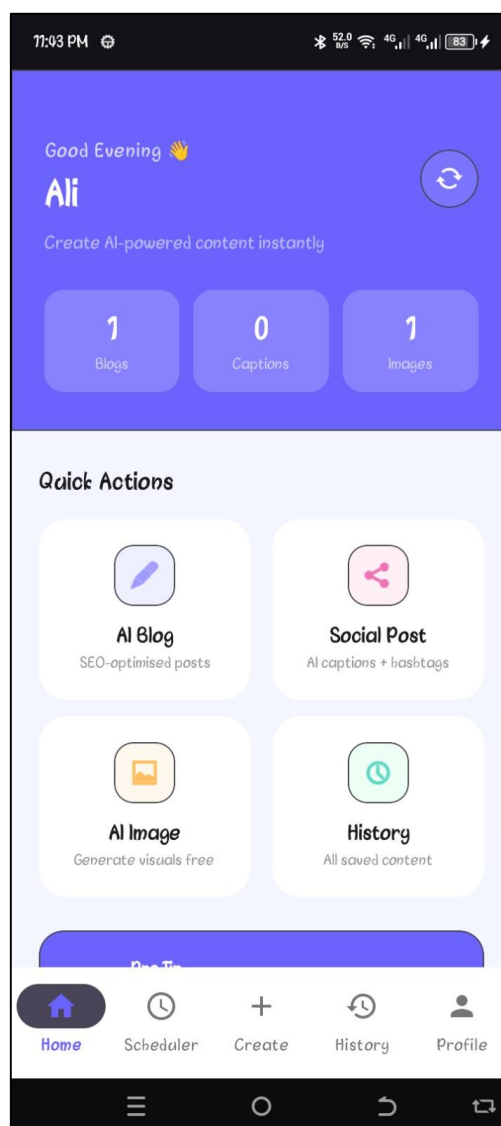


Figure 14: Dashboard Screen

4.10.4 Blog Generation Screen

This screen features input fields for the blog topic and a tone selector (Professional, Casual, Persuasive). A prominent Generate button initiates the creation process. The generated blog post is displayed in a scrollable text area below, with action buttons for Copy, Share, and Humanize. A loading indicator appears during generation.

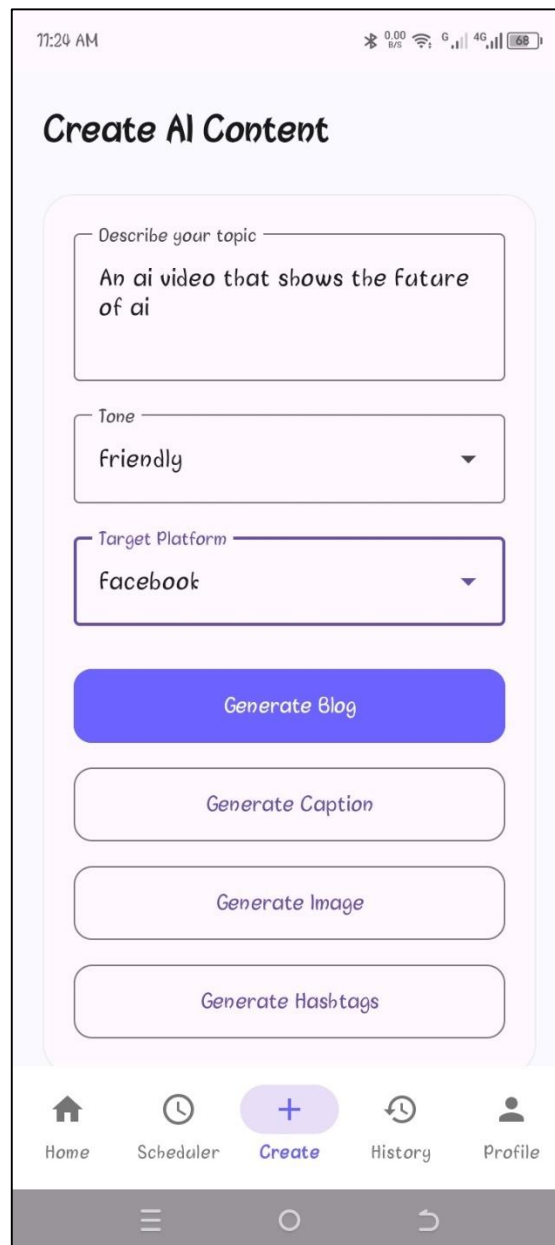


Figure 15: Blog Generation Screen

4.10.5 Social Media Caption Screen

This screen allows users to generate platform-specific social media content. It includes a platform selector (Instagram, LinkedIn, Twitter/X), a topic input field, and a Generate button. The output area displays the generated caption along with relevant hashtags, formatted for easy copying and sharing.

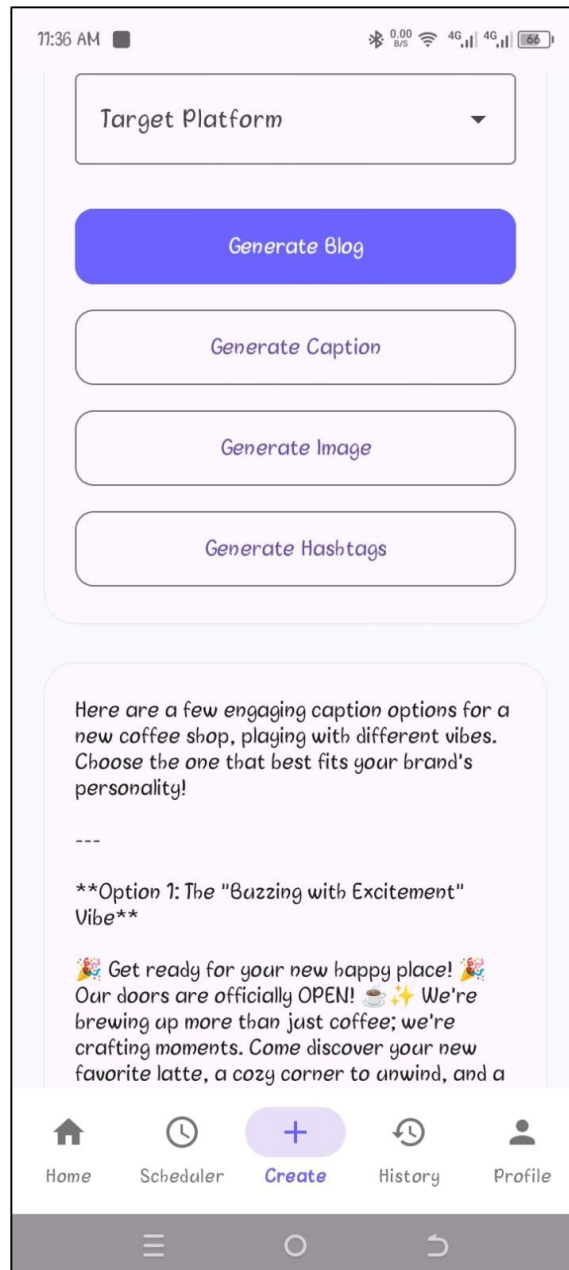


Figure 16: Social Media Caption Screen

4.10.6 Image Generation Screen

Users input a detailed text prompt describing the desired image. A visible usage tracker shows the daily free image generation limit (e.g., 'Free images today: 2 / 20'). The generated image is displayed prominently in the center of the screen, with action buttons to Save to history, Download to gallery, or Share via other apps.

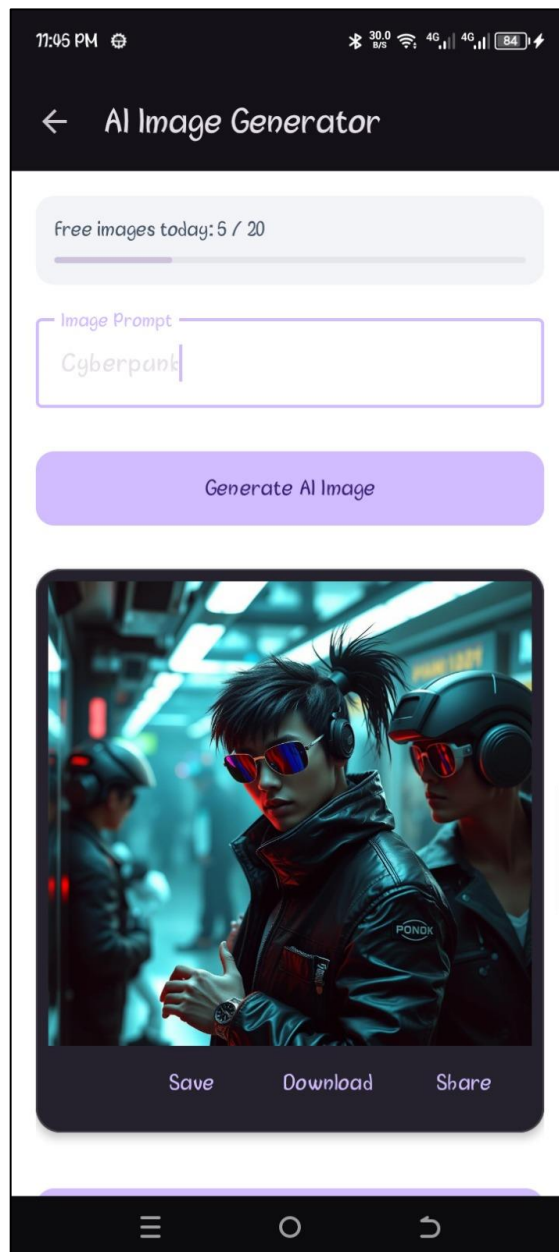


Figure 17: Image Generation Screen

4.10.7 History Screen

This screen displays a scrollable list of all previously generated content. Each item shows a type badge (Blog, Caption, or Image), the topic label, generation timestamp, and action buttons to copy text content or delete the item. A search bar at the top allows filtering, and tabs enable filtering by content type. The list is populated from Cloud Firestore in real-time.

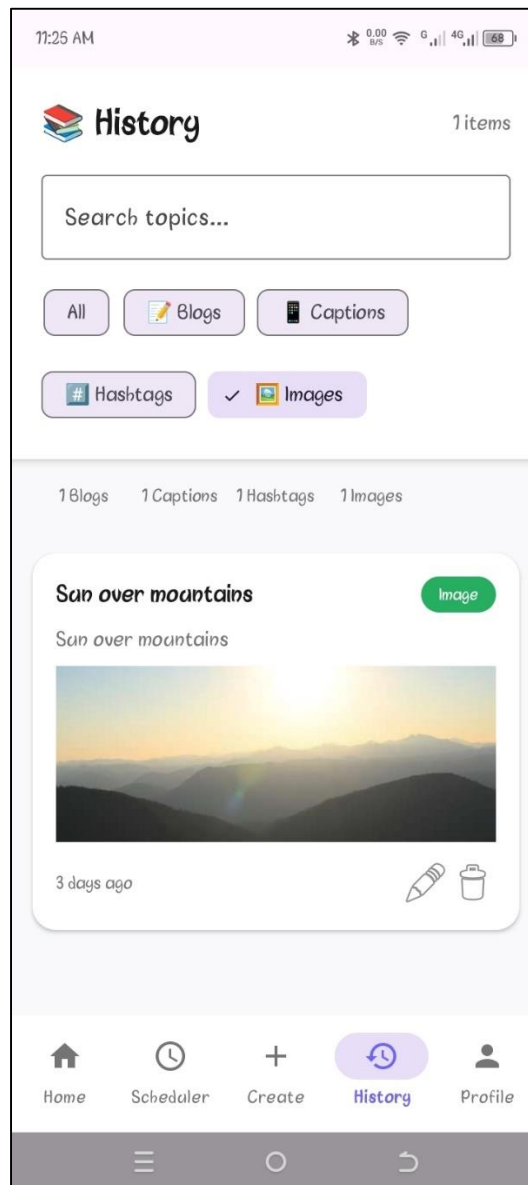


Figure 18: History Screen

CHAPTER 5

SYSTEM TESTING

5.1 Testing Introduction

System testing is an essential stage in software development, focusing on assessing the fully integrated product to confirm it meets all requirements. For the Agentic Marketer, the testing process was carefully designed and carried out to detect defects, verify features, and validate performance, reliability, and usability. This chapter details the testing strategy, methods used, specific test cases, and the results, ensuring the application's quality and readiness for deployment.

5.2 Purpose of Testing

The main objectives of testing the Agentic Marketer were diverse. Firstly, requirements verification aimed to confirm that the application functions correctly and meets all specified functional and non-functional requirements. Secondly, defect detection sought to identify bugs, errors, or inconsistencies in the software before release. Thirdly, quality assurance focused on maintaining the application's overall quality, including its stability, performance, security, and usability. Fourth, risk mitigation involved identifying and fixing potential issues early to minimize post-release failures. Lastly, user satisfaction aims to ensure the delivery of a reliable, seamless product that offers a positive experience for users.

5.3 Objectives of Testing

Specific objectives guided the testing process. It confirmed the functional correctness of all AI content-generation features—including blog creation, social media integration, image generation—as well as authentication and history management. Performance testing evaluated the application's responsiveness during API calls and its resource efficiency. Usability testing checked that the user interface is easy to use and provides clear feedback. Reliability and stability tests ensured the app remained stable under various conditions, including network fluctuations, without crashing or data loss. Security validation confirmed that user data and API keys are handled securely and that authentication systems are robust.

5.4 Testing Techniques

Both Black Box and White Box testing techniques were employed to provide comprehensive test coverage. This dual approach allowed for validation of both external behavior and internal logic.

5.4.1 Black Box Testing

Black Box Testing concentrates on the software's external behavior without examining its internal code. For Agentic Marketer, we used equivalence partitioning on input data—for instance, splitting email inputs into valid and invalid format groups for the login screen. Boundary value analysis tested text prompt lengths by evaluating the minimum and maximum acceptable sizes for blog topics and image descriptions. Decision table testing was employed to analyze the content generation process, covering various scenarios such as successful API responses, network failures, empty inputs, and rate-limit conditions.

5.4.2 White Box Testing

White Box Testing involves analyzing the internal workings of the software. For Agentic Marketer, we focused on path coverage in the ViewModel and Repository layers, ensuring that every independent path in key code sections was executed at least once. Loop testing verified the accuracy of repeated processes, especially for the RecyclerView adapter managing content lists. Data flow testing tracked how data moves through the system to confirm proper initialization, use, and storage of content objects from the UI, through the ViewModel and Repository, and finally to Cloud Firestore.

5.5 Black Box Test Cases

The following table presents the comprehensive black box test cases designed to cover the primary user interactions and functional requirements:

Test Case ID	Feature Tested	Input	Expected Output	Actual Output	Result
BB-001	User Registration	Valid email, strong password	User Registered Successfully	User Registered Successfully	Pass
BB-002	User Login	Valid email, correct password	User Logged In, Dashboard displayed	User Logged In, Dashboard displayed	Pass
BB-003	User Login	Valid email, incorrect password	Invalid Credentials error message	Invalid Credentials error message	Pass
BB-004	User Login	Unregistered email address	User Not Found error message	User Not Found error message	Pass
BB-005	Blog Generation	Topic: 'Future of AI in Marketing', Tone: Professional	SEO-optimized blog post generated	Detailed blog post generated and displayed	Pass
BB-006	Blog Generation	Empty topic field	Please enter a topic error message	Please enter a topic error message	Pass
BB-007	Social Caption Generation	Product description input	Caption and hashtags generated	Caption with relevant hashtags displayed	Pass
BB-008	Image Generation	Prompt: 'A futuristic'	Image	High-quality	Pass

Test Case ID	Feature Tested	Input	Expected Output	Actual Output	Result
		smartphone on a sleek desk'	generated and displayed	image generated and displayed	
BB-009	Image Generation	Empty prompt field	Please enter a description error	Please enter a description error	Pass
BB-010	View History	Navigate to the History screen	List of past content displayed	List displayed with correct items	Pass
BB-011	Delete History Item	Select the item and confirm the deletion	Item removed from list	Item removed from the list and Firestore	Pass
BB-012	No Internet	Trigger blog generation without a connection	Connection Error dialog shown	Connection Error dialog displayed	Pass

Table 7: Black Box Test Cases

5.6 White Box Test Cases

White box testing focused on the internal logic of key components, particularly the Content Repository and FirestoreManager classes, which handle interactions with external AI APIs and Firebase.

5.6.1 White Box Test Case: API Error Handling in Content Repository

The following Kotlin code snippet from ContentRepository.kt was analysed for path coverage:

```
suspend fun generateText(prompt: String): Result<String> {

    return try {

        val response = apiService.generateText(GeminiRequest(prompt))

        if (response.isSuccessful) {

            Result.success(response.body()?.text ?: "")

        } else {

            when (response.code()) {

                401 -> Result.failure(ApiException("Unauthorized access"))

                429 -> Result.failure(ApiException("Rate limit exceeded"))

                500 -> Result.failure(ApiException("Server error"))

                else -> Result.failure(ApiException("Unknown API error"))

            }

        }

    } catch (e: Exception) {

        Result.failure(e)

    }

}
```

The six independent paths through this function are:

Path 1 (Success): `response.isSuccessful = true` — returns `Result.success` with generated text.

Path 2 (401 Error): `response.isSuccessful = false`, `code = 401` — returns `Result.failure` with 'Unauthorized access'.

Path 3 (429 Error): `response.isSuccessful = false`, `code = 429` — returns `Result.failure` with 'Rate limit exceeded'.

Path 4 (500 Error): `response.isSuccessful = false`, `code = 500` — returns `Result.failure` with 'Server error'.

Path 5 (Other HTTP Error): `response.isSuccessful = false`, code is not 401/429/500 — returns `Result.failure` with 'Unknown API error'.

Path 6 (Network Exception): Exception thrown (e.g., `IOException`) — returns `Result.failure` with the exception.

ID	Condition	Input (Simulated)	Expected Output	Actual Output	Result
WB-001	Successful API Call	<code>isSuccessful = true,</code> <code>body = {text: 'blog'}</code>	<code>Result.success('blog')</code>	<code>Result.success('blog')</code>	Pass
WB-002	HTTP 401 Error	<code>isSuccessful = false,</code> <code>code = 401</code>	<code>Result.failure(ApiException('Unauthorized access'))</code>	<code>Result.failure(ApiException('Unauthorized access'))</code>	Pass
WB-003	HTTP 429 Error	<code>isSuccessful = false,</code> <code>code = 429</code>	<code>Result.failure(ApiException('Rate limit exceeded'))</code>	<code>Result.failure(ApiException('Rate limit exceeded'))</code>	Pass
WB-004	Network Exception	<code>IOException</code> thrown	<code>Result.failure(IOException)</code>	<code>Result.failure(IOException)</code>	Pass

Table 8: White Box Test Cases — API Error Handling

5.6.2 White Box Test Case: Firestore Data Persistence

The following Kotlin code snippet from `FirestoreManager.kt` was analyzed for path coverage:

```
suspend fun saveContent(content: GeneratedContent): Result<Unit> {
    return try {
        firestore.collection("users").document(content.userId)
            .collection("generated_content").add(content).await()
        Result.success(Unit)
    } catch (e) {
        Result.failure(e)
    }
}
```

```

    } catch (e: Exception) {

        Result.failure(e)

    }
}

suspend fun getContents(userId: String): Result<List<GeneratedContent>> {
    return try {

        val snapshot = firestore.collection("users").document(userId)

            .collection("generated_content").get().await()

        val contents = snapshot.toObject(GeneratedContent::class.java)

        Result.success(contents)

    } catch (e: Exception) {

        Result.failure(e)

    }
}

```

ID	Condition	Input	Expected Output	Actual Output	Result
WB-005	Save Content Success	Valid GeneratedContent object	Result.success(Unit)	Result.success(Unit)	Pass
WB-006	Retrieve Content Success	Valid userId with existing content	Result.success(List<GeneratedContent>)	Result: success with the correct list	Pass
WB-007	Retrieve Empty Content	Valid userId with no content	Result.success(emptyList())	Result: success with empty list	Pass

Table 9: White Box Test Cases — Firestore Data Persistence

5.7 Testing Results and Discussion

The comprehensive testing phase for Agentic Marketer yielded positive results, with the application demonstrating high levels of functional correctness, stability, and usability. All critical functional requirements were met, and the non-functional aspects performed within acceptable parameters.

All 12 black-box test cases passed, confirming that user authentication, content generation (blog, social, image), history management, and error-handling functionalities work as expected. Minor UI glitches were identified and rectified during the Increment 2 iteration, particularly in the history list scroll behavior and the timing of the loading indicator.

White box testing, particularly for API integration and data persistence logic, ensured robust error handling and data integrity. The `ContentRepository` successfully managed various API responses, including the retry mechanism for Hugging Face rate limits, and the `FirestoreManager` reliably stored and retrieved user content.

Performance testing showed that text generation typically completed within 5–7 seconds and image generation within 15–20 seconds on a stable internet connection. These results meet the specified non-functional requirements. The Material Design 3 UI received positive feedback during informal testing sessions, with users praising the intuitive layout and smooth animations. Firebase Authentication and Firestore security rules were validated to ensure data privacy and secure operations.

CHAPTER 6

CONCLUSION & FUTURE WORK

6.1 Conclusion

The Agentic Marketer project has developed a practical AI-powered Android app that meets a real need in digital marketing. It combines text creation with Google Gemini 1.5 Pro, image generation with Hugging Face FLUX.1-schnell, and reliable data storage with Firebase. This results in a tool that effectively supports small businesses and individual marketers in producing content more efficiently and affordably compared to current options.

This project's technical accomplishments go beyond basic features. Using MVVM architecture with Kotlin created a robust base for maintainable and testable code. Managing various external APIs—each with unique authentication methods, rate limits, and response structures—necessitated careful abstraction via the Repository pattern, which we successfully applied. Choosing Retrofit for type-safe HTTP requests and Cloud Firestore for real-time data sync was a wise decision, as both technologies proved dependable during development and testing.

The Incremental Development Model served us well. By delivering working functionality at the end of each increment, we were able to gather feedback early and adjust our approach. During Increment 2, we discovered that Firestore's default indexing was insufficient for efficient history queries, which led us to implement composite indexes. This lesson will inform our future work with NoSQL databases. During Increment 3, API rate limits from Hugging Face's free tier required implementing a retry mechanism with exponential backoff, which added resilience to the image generation flow.

From a user experience perspective, the Material Design 3 interface with Lottie animations provides a polished, modern feel that competes with commercial applications. The visible usage tracker for image generation, the centered Create button in the bottom navigation, and the humanize feature for blog posts all came from thinking about what actual users would need — not just what was technically interesting to build.

Looking ahead, Agentic Marketer stands as a meaningful contribution to the field of mobile-first AI marketing tools. It demonstrates that sophisticated AI capabilities can be packaged into accessible, free-to-use mobile applications that genuinely help resource-constrained businesses compete in a content-driven digital economy. The foundation we

have built — both the codebase and the architecture — provides a solid platform for the enhancements described in the following section.

6.2 Future Work

While Agentic Marketer delivers a powerful suite of AI-driven content creation tools, the dynamic nature of AI and digital marketing presents numerous opportunities for future enhancements. The following areas are identified as potential avenues for development:

6.2.1 Advanced Social Media Integration

The next best step is to enable direct API integration for posting on major social media platforms such as Instagram, LinkedIn, Facebook, and X (formerly Twitter). This involves connecting to their developer APIs, allowing users to publish content directly from the app, eliminating manual copying and pasting. Furthermore, adding a scheduling feature would let users plan and set posts in advance, boosting their social media reach and engagement.

6.2.2 Video Content Generation

As AI video generation technology advances, integrating services like OpenAI Sora or Google Lumiere could enable users to produce short, engaging marketing videos from text prompts or images. This would significantly broaden the variety of content formats the app can support. Adding basic editing features within the app—such as text overlays, background music, and simple transitions—would also enhance the practicality and appeal of the videos created.

6.2.3 Analytics and Performance Tracking

Creating an analytics dashboard in the app would enable users to monitor how their published content performs. Key metrics such as engagement rates, reach, clicks, and conversions would provide valuable insights. AI-powered recommendation algorithms could analyse content performance data and suggest enhancements to upcoming content strategies, tone, and topics.

6.2.4 Multimodal Input and Generation

Voice-to-content generation enables users to input voice commands to create content, using speech-to-text technology to convert spoken ideas into marketing text or image prompts. The image-to-text and image-to-image features allow users to analyze existing images and produce descriptive descriptions, or modify images into different styles or variations.

6.2.5 Personalization and Brand Consistency

Brand voice customization would allow users to define and save their brand's unique voice, tone, and style guidelines. The AI models would then be prompted to adhere to these guidelines, ensuring greater brand consistency across all generated content. User persona management would enable the creation of different marketing personas, allowing the AI to generate content tailored to specific target audiences.

6.2.6 Advanced Collaboration Features

Team workspaces would introduce collaboration features, allowing multiple users to work on projects, share content, and manage marketing campaigns collectively within the application. Approval workflows would enable supervisors or clients to review and approve content before publication, adding a layer of quality control for professional teams.

REFERENCES

- [1] Google, "Gemini API Documentation," Google AI for Developers, 2024. [Online]. Available: <https://ai.google.dev/docs>
- [2] Hugging Face, "FLUX.1-schnell Model Card," Hugging Face, 2024. [Online]. Available: <https://huggingface.co/black-forest-labs/FLUX.1-schnell>
- [3] T. Brown et al., "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 1877-1901.
- [4] U.S. Chamber of Commerce, "AI and Small Business: A Report on the Impact of Artificial Intelligence," 2024. [Online]. Available: <https://www.uschamber.com>
- [5] Marketermilk, "Marketing Trends 2026: The Rise of Agentic AI," Marketermilk, 2026. [Online]. Available: <https://marketermilk.com>
- [6] Forbes, "AI and Small Business Marketing: Challenges and Opportunities," Forbes, May 2026. [Online]. Available: <https://www.forbes.com>
- [7] McKinsey & Company, "Reinventing Marketing with Agentic AI," McKinsey Digital, 2024. [Online]. Available: <https://www.mckinsey.com>
- [8] Deloitte Digital, "Agentic AI in Marketing: The Next Frontier," Deloitte Insights, 2024. [Online]. Available: <https://www.deloitte.com>
- [9] Google, "Firebase Authentication Documentation," Firebase, 2024. [Online]. Available: <https://firebase.google.com/docs/auth>
- [10] Google, "Cloud Firestore Documentation," Firebase, 2024. [Online]. Available: <https://firebase.google.com/docs/firestore>
- [11] Square Inc., "Retrofit HTTP Client," Square Open Source, 2024. [Online]. Available: <https://square.github.io/retrofit>
- [12] I. Sommerville, Software Engineering, 10th ed., Pearson, 2016.
- [13] Google, "Material Design 3 Guidelines," Material.io, 2024. [Online]. Available: <https://m3.material.io>
- [14] OpenRouter, "OpenRouter API Documentation: Auto-Routed LLM Access," 2024. [Online]. Available: <https://openrouter.ai/docs>